
Software Requirements Specification

for

Smart Vehicle Service & Management System

Version 1.0

Prepared by Hari Kumar L

Department of Computer Science and Engineering

15-10-2026

Table of Contents

Table of Contents	ii
Revision History	iii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	2
1.5 References.....	2
2. Overall Description	2
2.1 Product Perspective.....	2
2.2 Product Functions	3
2.3 User Classes and Characteristics.....	3
2.4 Operating Environment.....	7
2.5 Design and Implementation Constraints	7
2.6 User Documentation	8
2.7 Assumptions and Dependencies.....	8
3. External Interface Requirements.....	8
3.1 User Interfaces	8
3.2 Hardware Interfaces	9
3.3 Software Interfaces	9
3.4 Communications Interfaces.....	9
4. System Features.....	9
4.1 User Authentication and Role-Based Access Control.....	9
4.2 User and Profile Management.....	10
4.3 Vehicle Management	10
4.4 Service Provider Search and Selection.....	11
4.5 Service Booking and Appointment Management	11
4.6 Service Request and Technician Assignment	11
4.7 Vehicle Inspection and Service Management	12
4.8 Inventory and Spare Parts Management.....	12
4.9 Service Progress and Quality Management.....	13
4.10 Service History and Notifications	13
4.11 Reports and Administrative Management.....	14
5. Other Nonfunctional Requirements	14
5.1 Performance Requirements	14
5.2 Safety Requirements	14
5.3 Security Requirements	15
5.4 Software Quality Attributes	15
5.5 Business Rules	15
6. Other Requirements.....	16
Appendix A: Glossary	16
Appendix B: Analysis Models	17
Appendix C: To Be Determined List.....	40

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Smart Vehicle Service & Management System, Version 1.0. The system is a web-based software application developed to automate and streamline vehicle service management activities.

The system provides a centralized platform for vehicle registration, service provider selection, service booking, appointment management, technician assignment, vehicle inspection, spare-parts management, service progress tracking, quality checking, service completion, service history, notifications, and report generation.

This document defines the complete scope of the system and serves as the primary reference for the design, development, testing, implementation, and maintenance of the Smart Vehicle Service & Management System.

1.2 Document Conventions

This document follows the IEEE-style Software Requirements Specification structure. The following conventions apply: (1) Requirements are written using the keyword SHALL for mandatory requirements and SHOULD for recommended features. (2) Each functional requirement is assigned a unique identifier in the format REQ-XX. (3) Non-functional requirements are described under the appropriate NFR sections. (4) Section headings follow a hierarchical decimal numbering system. (5) Bold text is used to highlight critical terms and important definitions. (6) User roles such as Administrator, Customer, Service Provider, Technician, and Inventory Manager are explicitly identified throughout the document. (7) Higher-level requirements are assumed to apply to their related child requirements unless otherwise stated. (8) Vehicle, customer, service, inventory, and appointment information referenced in the system shall follow standardized formats to ensure consistency and ease of maintenance.

1.3 Intended Audience and Reading Suggestions

This SRS document is intended for the following audiences: (1) Software Developers and Engineers – responsible for designing and implementing the system; they should read all sections thoroughly. (2) Project Managers and Project Guides – responsible for project planning, scheduling, monitoring, and progress tracking; they should focus on Sections 1, 2, and 5. (3) System Testers and QA Engineers – responsible for verifying that the system satisfies all specified requirements; they should focus on Sections 3 and 4. (4) Administrators – responsible for managing users, staff approvals, system information, and reports; they should read Sections 1, 2, and 4. (5) Service Providers and Technicians – responsible for handling service requests, inspections, technician assignments, and service activities; they should focus on the relevant functionalities described in Section 4. (6) Inventory Managers – responsible for spare-parts management, stock monitoring, supplier information, and parts issuing; they should focus on the inventory-related requirements. (7) Customers – users who register vehicles, select service providers, book services, track service progress, and view service history; they should focus on the relevant user functionalities described in Section 4. It is recommended that all readers begin with Section 1 and Section 2 to gain a comprehensive understanding of the product before proceeding to the detailed requirements.

1.4 Product Scope

The Smart Vehicle Service & Management System is a role-based web application designed to digitize and manage essential vehicle service operations.

The primary goal of the system is to provide a centralized and organized platform for managing the complete vehicle service lifecycle. The system reduces manual record keeping, improves service coordination, provides better visibility of service progress, and maintains service history.

The system enables customers to register their vehicles, search and select service providers based on location, book service appointments, and monitor service progress. Service Providers can manage booking requests, accept or reject service requests, assign technicians, and monitor ongoing services. Technicians can inspect vehicles, record inspection details, perform service activities, update service progress, and request required spare parts. Inventory Managers can maintain spare-part records, monitor stock levels, manage suppliers, identify low-stock items, and issue required parts. Administrators can manage users, approve staff registration requests, monitor system activities, and generate reports.

The main objectives of the system include: (1) providing a unified platform for customers and service staff; (2) managing vehicle and customer information; (3) simplifying service provider selection and booking; (4) managing service appointments and service requests; (5) assigning technicians to service tasks; (6) recording vehicle inspection and service information; (7) managing spare parts and inventory; (8) tracking service progress; (9) maintaining service history; (10) providing maintenance reminders and notifications; and (11) generating reports for administrative monitoring.

1.5 References

The following references were consulted during the preparation of this SRS: (1) IEEE Std 830-1998, "Recommended Practice for Software Requirements Specifications," IEEE, 1998. (2) Sommerville, I., "Software Engineering," 10th Edition, Pearson Education. (3) Pressman, R. S., "Software Engineering: A Practitioner's Approach," McGraw-Hill. (4) React.js Documentation. (5) Vite Documentation. (6) Supabase Documentation. (7) PostgreSQL Documentation. (8) HTML5 Documentation, World Wide Web Consortium (W3C). (9) CSS Documentation, World Wide Web Consortium (W3C). (10) JavaScript Documentation, Mozilla Developer Network (MDN).

2. Overall Description

2.1 Product Perspective

The Smart Vehicle Service & Management System is a web-based application developed to replace fragmented and manual vehicle service management processes.

The system consolidates customer, vehicle, booking, service, technician, inventory, and service-history information into a centralized platform accessible through a web browser. The system provides role-specific interfaces for Administrator, Customer, Service Provider, Technician, and Inventory Manager.

The application is designed using a frontend architecture based on React.js and Vite. Supabase can be used for authentication and PostgreSQL-based data storage. The application is structured using reusable components so that a backend REST API such as Spring Boot can be integrated in the future if required.

A high-level system architecture may include: (1) Client Layer – web browser and responsive user interface. (2) Application Layer – React.js application and business interface logic. (3) Data and Service Layer – Supabase authentication, PostgreSQL database, and related services. The system can be extended in the future with additional vehicle service features, notification services, analytics, and backend APIs.

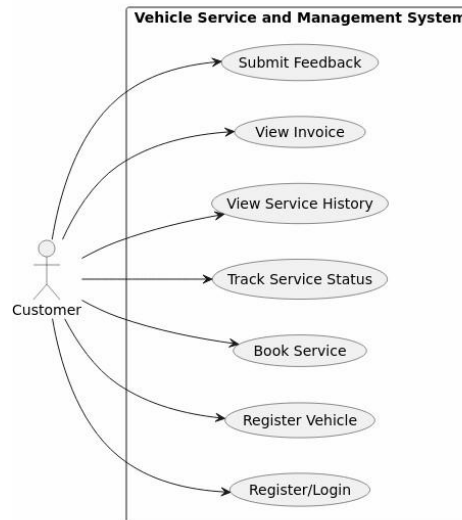
2.2 Product Functions

The Smart Vehicle Service & Management System shall provide the following major functions: (1) User Authentication and Role-Based Access Control – secure login, logout, registration, and role-specific permissions. (2) User and Profile Management – management of customer and staff profiles and staff approval requests. (3) Vehicle Management – registration, viewing, and updating of customer vehicle information. (4) Service Provider Search and Selection – finding and selecting service providers based on location and service requirements. (5) Service Booking Management – creating, viewing, accepting, rejecting, and managing service appointments. (6) Technician Assignment – assigning technicians to accepted service requests. (7) Vehicle Inspection and Service Management – recording inspection information, diagnosis, service tasks, and work records. (8) Inventory Management – maintaining spare-part information, stock levels, suppliers, and low-stock information. (9) Spare-Part Request and Issuing – requesting and issuing parts required for vehicle service. (10) Service Progress and Quality Management – tracking service status and performing quality checks before completion. (11) Service History Management – maintaining records of completed vehicle services. (12) Notifications and Reminders – providing relevant maintenance and service notifications. (13) Report Generation – providing reports related to users, vehicles, bookings, services, technicians, and inventory. (14) Administrative Management – providing administrative control over system users and operations.

2.3 User Classes and Characteristics

The system serves five distinct user classes: (1) Administrator – highest privilege level; manages users, staff approval requests, system activities, records, and reports. The Administrator uses the system regularly and requires access to administrative functions. (2) Customer – registers vehicles, searches for service providers, books service appointments, monitors service progress, receives notifications, and views service history. Customers generally access the system when vehicle service is required. (3) Service Provider – manages service bookings, accepts or rejects requests, assigns technicians, and monitors service activities. Service Providers are frequent operational users. (4) Technician – performs vehicle inspection and service activities, records work details, requests spare parts, and updates service progress. Technicians use the system during service operations. (5) Inventory Manager – manages spare-part inventory, stock quantities, supplier information, low-stock items, and parts requested by technicians. Inventory Managers use the system regularly for inventory operations.

1. Customer:



Use Case: Book Service

Primary Actor: Customer

Secondary Actor: System

Precondition: Customer has registered, logged in, and registered at least one vehicle.

Postcondition: A service appointment is successfully booked and stored in the system.

Trigger: Customer selects the Book Service option.

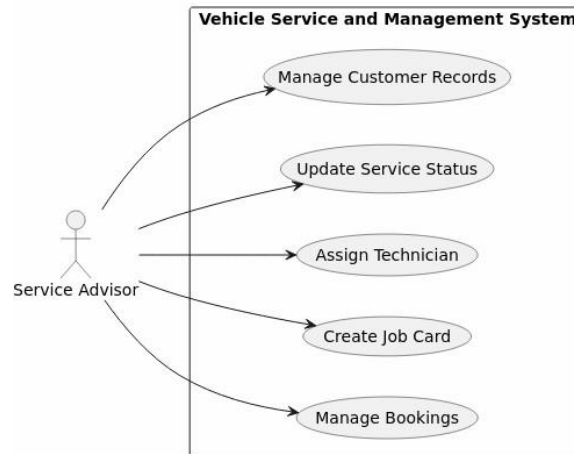
Main Access Scenario:

- Register/Login.
- Select a registered vehicle.
- Choose service type.
- Select preferred date and time.
- Confirm the booking.
- Receive booking confirmation.

Exception Scenario:

- Invalid login credentials.
- No service slots available.
- Vehicle not registered.

2. Service Advisor:



Use Case: Create Job Card

Primary Actor: Service Advisor

Secondary Actor: System

Precondition: Customer has booked a service and the Service Advisor is logged in.

Postcondition: A job card is created and assigned for servicing.

Trigger: Service Advisor selects a booked vehicle for servicing.

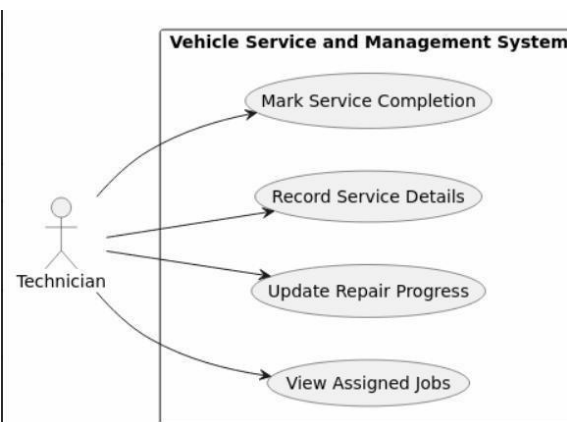
Main Access Scenario:

- Log in.
- View booked appointments.
- Verify customer and vehicle details.
- Create a job card.
- Assign the job to a technician.
- Save the job card.

Exception Scenario:

- Invalid login.
- Customer booking not found.
- Required details are missing.

3. Technician:



Use Case: Update Repair Progress

Primary Actor: Technician

Secondary Actor: System

Precondition: Technician is logged in and has an assigned job card.

Postcondition: Repair progress is updated successfully.

Trigger: Technician starts or continues the assigned repair work.

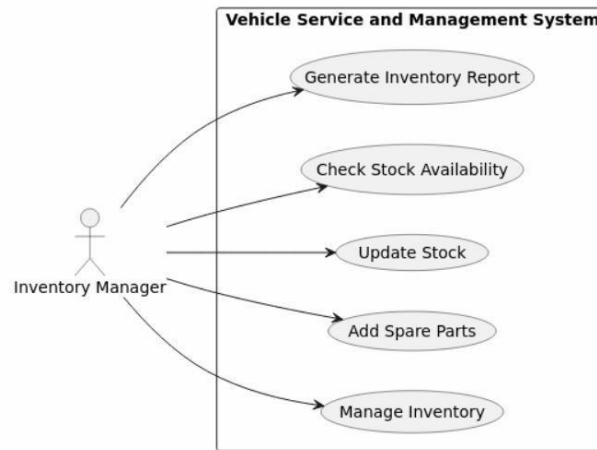
Main Access Scenario:

- Log in.
- View assigned jobs.
- Open the assigned job card.
- Update repair progress.
- Save repair information.
- Notify the Service Advisor.

Exception Scenario:

- Invalid login.
- No assigned jobs available.
- System or network failure.

4. Inventory Manager:



Use Case: Manage Inventory

Primary Actor: Inventory Manager

Secondary Actor: System

Precondition: Inventory Manager has registered and logged in.

Postcondition: Spare parts inventory is updated successfully.

Trigger: Inventory Manager selects the Manage Inventory option.

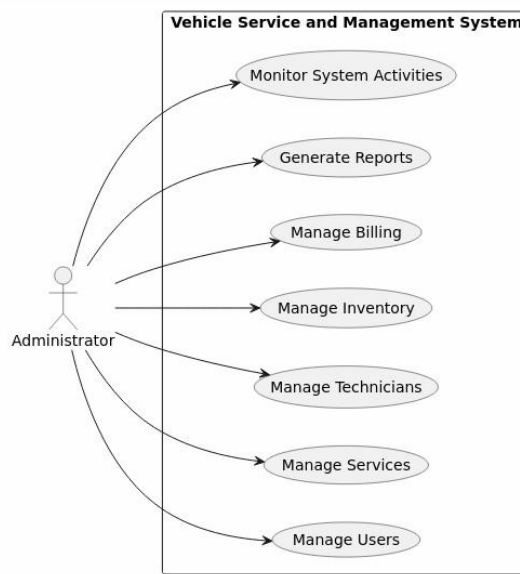
Main Access Scenario:

- Log in.
- View spare parts inventory.
- Add new spare parts.
- Update stock quantity.
- Remove unavailable items.
- Save inventory changes.

Exception Scenario:

- Invalid login.
- Duplicate spare part entry.
- Database or server error.

5. Admin:



Use Case: Manage Users

Primary Actor: Admin

Secondary Actor: System

Precondition: Admin has registered and logged in.

Postcondition: User accounts and system records are managed successfully.

Trigger: Admin logs in or selects the Manage Users function.

Main Access Scenario:

- Log in.
- View all user accounts.
- Add new users.
- Update user details.
- Delete or deactivate user accounts.
- Save changes.

Exception Scenario:

- Invalid login.
- User record not found.
- System/database error.

2.4 Operating Environment

The Smart Vehicle Service & Management System shall operate in the following environment: (1) Client Side: Modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. (2) Operating Systems: Windows, macOS, Linux, Android, and iOS through supported browsers. (3) Development Environment: Visual Studio Code or equivalent development environment. (4) Frontend: React.js with Vite, JavaScript, React Router, and CSS. (5) Database and Backend Services: Supabase PostgreSQL and Supabase Authentication. (6) Network: Stable internet connection for hosted services. (7) Security: HTTPS shall be used for deployed client-server communication.

2.5 Design and Implementation Constraints

The following constraints shall apply to the development of the system: (1) The frontend shall be developed using React.js and Vite. (2) React Router shall be used for application navigation. (3) JavaScript shall be

used for application logic. (4) CSS shall be used for interface styling. (5) The application shall use reusable and modular React components. (6) Supabase shall be used for authentication and PostgreSQL-based data storage where implemented. (7) Role-Based Access Control shall be enforced throughout the application. (8) The Administrator account shall be controlled and there shall be no open Administrator signup. (9) Staffusers such as Service Provider, Technician, and Inventory Manager shall submit signup requests that require Administrator approval before activation. (10) Customer signup shall not require email verification. (11) The user interface shall be responsive and compatible with modern browsers. (12) All user inputs shall be validated before processing. (13) The system shall maintain consistent relationships between customers, vehicles, bookings, services, technicians, and inventory records. (14) The project shall be demonstrable in a local development environment.

2.6 User Documentation

The following user documentation may be delivered along with the system: (1) User Manual – guide covering the major functionalities for customers and staff. (2) Administrator Guide – instructions for user management, staff approval, monitoring, and reports. (3) Customer Guide – instructions for vehicle registration, service provider selection, booking, tracking, and service history. (4) Staff Guide – instructions for Service Providers, Technicians, and Inventory Managers. (5) Quick Reference Guide – concise instructions describing common operations. (6) Installation Guide – instructions for setting up the frontend and required services.

2.7 Assumptions and Dependencies

The following assumptions and dependencies apply to this project: Assumptions: (1) It is assumed that all users have basic computer or smartphone literacy. (2) It is assumed that customers enter accurate vehicle and service information. (3) It is assumed that Service Providers update booking decisions correctly. (4) It is assumed that Technicians update inspection and service records accurately. (5) It is assumed that Inventory Managers maintain correct stock information. (6) It is assumed that users have access to a stable internet connection when using hosted services. Dependencies: (1) The frontend depends on React.js and Vite. (2) Navigation depends on React Router. (3) Authentication depends on Supabase Authentication when implemented. (4) Persistent data storage depends on Supabase PostgreSQL when implemented. (5) Role-based data access depends on appropriate authorization and Row-Level Security configuration. (6) Future backend integration may depend on a Spring Boot REST API.

3. External Interface Requirements

3.1 User Interfaces

The user interface shall provide separate role-appropriate dashboards for Administrator, Customer, Service Provider, Technician, and Inventory Manager. Common controls include navigation menu, dashboard, search, forms, tables, filters, status indicators, confirmation dialogs, validation messages, notifications, profile/account controls, and logout. Customer screens shall support vehicle registration, vehicle management, service provider search, service booking, appointment viewing, service tracking, service history, and notifications. Service Provider screens shall support booking management, accept/reject operations, technician assignment, and service monitoring. Technician screens shall support assigned service viewing, vehicle inspection, service work recording, parts requests, and progress updates. Inventory Manager screens shall support spare-part management, stock management, supplier management, low-stock monitoring, and parts issuing. Administrator screens shall support user management, staff approval, system monitoring, reports, and administrative functions. Error messages shall clearly identify the problem and provide a suitable corrective action.

3.2 Hardware Interfaces

The system shall support desktop computers, laptops, smartphones, and tablets through a modern web browser. No specialized hardware is required for normal operation. Client devices shall communicate with the application through an internet or network connection. The system shall provide a responsive interface suitable for different screen sizes.

3.3 Software Interfaces

The application shall use React.js and Vite for frontend development. React Router shall be used for navigation between application pages. JavaScript shall be used for application logic, validation, and user interaction, and CSS shall be used for styling and responsive layouts. Supabase shall provide authentication and PostgreSQL database services where implemented. Supabase Authentication shall be used to manage user authentication and sessions. Row-Level Security shall be used to control access to database records based on user roles where applicable. The application shall be designed so that a future Spring Boot REST API can be integrated for backend services.

3.4 Communications Interfaces

All browser-to-server or browser-to-cloud-service communication shall use HTTPS when deployed. Application requests and responses shall use standard HTTP methods and structured data such as JSON where applicable. Authentication and session information shall be protected during transmission. Network failures, timeouts, and unavailable services shall produce user-friendly error messages. Communication failures shall not result in inconsistent vehicle, booking, service, or inventory records.

4. System Features

4.1 User Authentication and Role-Based Access Control

4.1.1 Description and Priority

Priority: High. Provides secure registration, login, logout, authentication, and role-based access for Administrator, Customer, Service Provider, Technician, and Inventory Manager.

4.1.2 Stimulus/Response Sequences

- *User enters username/email and password and selects Login.*
- *System validates the entered credentials.*
- *If the credentials are invalid, the system displays an appropriate error message.*
- *If authentication is successful, the system identifies the assigned role.*
- *System redirects the user to the appropriate dashboard.*
- *Unauthorized users are denied access to restricted functions.*
- *Staff accounts remain inactive until approved by the Administrator.*

4.1.3 Functional Requirements

REQ-1: The system shall allow registered users to log in using valid authentication credentials.

REQ-2: The system shall validate required login fields and display an error for incomplete or invalid input.

REQ-3: The system shall provide role-based access for Administrator, Customer, Service Provider, Technician, and Inventory Manager.

REQ-4: The system shall prevent unauthorized users from accessing restricted modules and functions.

REQ-5: The system shall allow authenticated users to securely log out of the system.

4.2 User and Profile Management

4.2.1 Description and Priority

Priority: High. Allows authorized users to manage user accounts and profiles and allows the Administrator to manage staff approval requests.

4.2.2 Stimulus/Response Sequences

- *Administrator selects User Management.*
- *System displays available user records and staff requests.*
- *Administrator reviews pending staff registration requests.*
- *Administrator approves or rejects a request.*
- *System updates the request status.*
- *Users can view and update permitted profile information.*
- *Unauthorized users are prevented from modifying restricted information.*

4.2.3 Functional Requirements

REQ-6: The system shall allow authorized users to view and update permitted profile information.

REQ-7: The system shall allow the Administrator to create and manage authorized user records.

REQ-8: The system shall maintain staff signup requests with appropriate status.

REQ-9: The system shall activate Service Provider, Technician, and Inventory Manager accounts only after Administrator approval.

REQ-10: The system shall prevent unauthorized users from modifying restricted user information.

4.3 Vehicle Management

4.3.1 Description and Priority

Priority: High. Allows customers to register, view, and maintain information about their vehicles.

4.3.2 Stimulus/Response Sequences

- *Customer selects Vehicle Management.*
- *System displays the customer's registered vehicles.*
- *Customer enters vehicle details.*
- *System validates the entered information.*
- *Valid vehicle information is stored.*
- *Customer can view or update permitted vehicle information.*

4.3.3 Functional Requirements

REQ-11: The system shall allow customers to register vehicles.

REQ-12: The system shall store vehicle information including registration number, make, model, year, and relevant service information.

REQ-13: The system shall allow customers to update permitted vehicle information.

REQ-14: The system shall display registered vehicles belonging to the authenticated customer.

REQ-15: The system shall reject incomplete or invalid vehicle information before saving the record.

4.4 Service Provider Search and Selection

4.4.1 Description and Priority

Priority: High. Allows customers to search for and select a service provider based on location and service requirements.

4.4.2 Stimulus/Response Sequences

- *Customer selects a vehicle and service requirement.*
- *System displays available service providers.*
- *Customer applies location or other supported filters.*
- *System displays matching provider information.*
- *Customer selects a service provider.*
- *System associates the selected provider with the booking.*

4.4.3 Functional Requirements

REQ-16: The system shall allow customers to search for service providers using location-related information.

REQ-17: The system shall display relevant service provider information.

REQ-18: The system shall allow customers to select a service provider.

REQ-19: The system shall associate the selected provider with the customer's service booking.

REQ-20: The system shall display an appropriate message when no suitable service provider is available.

4.5 Service Booking and Appointment Management

4.5.1 Description and Priority

Priority: High. Manages service requests and appointments created by customers.

4.5.2 Stimulus/Response Sequences

- *Customer selects a registered vehicle.*
- *Customer selects a service provider.*
- *Customer enters service type and appointment details.*
- *System validates the booking information.*
- *System creates the booking request with an appropriate status.*
- *Service Provider reviews the request.*
- *Service Provider accepts or rejects the request.*
- *System updates the booking status.*

4.5.3 Functional Requirements

REQ-21: The system shall allow customers to create vehicle service booking requests.

REQ-22: The system shall validate required booking information before creating a booking.

REQ-23: The system shall maintain appointment date, time, vehicle, provider, and service details.

REQ-24: The system shall allow the Service Provider to accept or reject booking requests.

REQ-25: The system shall update and display the current booking status.

4.6 Service Request and Technician Assignment

4.6.1 Description and Priority

Priority: High. Coordinates accepted service requests and assigns technicians to perform the required service.

4.6.2 Stimulus/Response Sequences

- *Service Provider selects an accepted service request.*
- *System displays service and vehicle information.*
- *Service Provider selects an available technician.*
- *System records the technician assignment.*
- *Technician views the assigned service.*
- *System updates the service workflow.*

4.6.3 Functional Requirements

REQ-26: The system shall allow Service Providers to view accepted service requests.

REQ-27: The system shall allow authorized Service Providers to assign technicians.

REQ-28: The system shall maintain technician assignment information.

REQ-29: The system shall allow technicians to view their assigned service tasks.

REQ-30: The system shall prevent unauthorized users from changing technician assignments.

4.7 Vehicle Inspection and Service Management

4.7.1 Description and Priority

Priority: High. Supports vehicle inspection, diagnosis, service task recording, spare-part requests, and service progress updates.

4.7.2 Stimulus/Response Sequences

- *Technician opens an assigned service.*
- *Technician inspects the vehicle.*
- *Technician records inspection observations.*
- *System stores inspection information.*
- *Technician records diagnosis and required service tasks.*
- *If spare parts are required, Technician creates a parts request.*
- *Technician performs service activities.*
- *Technician updates the service progress.*

4.7.3 Functional Requirements

REQ-31: The system shall allow technicians to record vehicle inspection details.

REQ-32: The system shall maintain diagnosis and service work records.

REQ-33: The system shall allow technicians to update service progress.

REQ-34: The system shall allow technicians to request required spare parts.

REQ-35: The system shall associate inspection and service records with the corresponding vehicle and booking.

4.8 Inventory and Spare Parts Management

4.8.1 Description and Priority

Priority: High. Manages spare-part information, stock levels, supplier details, low-stock items, and issuing of parts required for vehicle service.

4.8.2 Stimulus/Response Sequences

- *Inventory Manager selects Inventory Management.*
- *System displays spare-part records and stock quantities.*
- *Inventory Manager adds or updates spare-part information.*
- *Technician submits a spare-parts request.*
- *Inventory Manager reviews the request.*
- *Inventory Manager issues available parts.*
- *System updates stock quantities and request status.*

4.8.3 Functional Requirements

REQ-36: The system shall allow the Inventory Manager to add new spare-part records.

REQ-37: The system shall store spare-part details including name, quantity, supplier, and stock status.

REQ-38: The system shall identify low-stock items according to configured thresholds.

REQ-39: The system shall allow the Inventory Manager to view and manage technician parts requests.

REQ-40: The system shall update inventory quantities after parts are issued.

4.9 Service Progress and Quality Management

4.9.1 Description and Priority

Priority: High. Tracks the progress of vehicle service from active work through quality checking and final completion.

4.9.2 Stimulus/Response Sequences

- *Technician updates the current service stage.*
- *System records the latest service status.*
- *Authorized users can view the service progress.*
- *Technician completes the required service activities.*
- *Vehicle is submitted for quality checking.*
- *Quality-check information is recorded.*
- *If the service passes the required check, it is marked completed.*
- *If issues are identified, the service is returned for required action.*

4.9.3 Functional Requirements

REQ-41: The system shall maintain the current status of each service request.

REQ-42: The system shall allow authorized users to view service progress.

REQ-43: The system shall provide a quality-check stage before final service completion.

REQ-44: The system shall record quality-check results.

REQ-45: The system shall mark a service as completed only after the required completion conditions are satisfied.

4.10 Service History and Notifications

4.10.1 Description and Priority

Priority: Medium. Maintains completed service history and provides relevant notifications and maintenance reminders.

4.10.2 Stimulus/Response Sequences

- *Customer selects Service History.*
- *System retrieves completed service records.*
- *System displays service details for the customer's vehicle.*
- *System generates or displays relevant maintenance reminders.*
- *User receives appropriate notifications.*

4.10.3 Functional Requirements

REQ-46: The system shall maintain a service history for each registered vehicle.

REQ-47: The system shall allow customers to view their permitted service history.

REQ-48: The system shall display service details associated with completed bookings.

REQ-49: The system shall support maintenance and periodic service reminders.

REQ-50: The system shall display relevant notifications to users.

4.11 Reports and Administrative Management

4.11.1 Description and Priority

Priority: High. Provides authorized users with management information and reports related to users, vehicles, bookings, services, technicians, and inventory.

4.11.2 Stimulus/Response Sequences

- *Administrator selects the required management or report option.*
- *System retrieves relevant data.*
- *Administrator selects available filters.*
- *System validates the requested parameters.*
- *System generates and displays the requested information.*
- *If sufficient data is unavailable, the system displays an appropriate message.*

4.11.3 Functional Requirements

REQ-51: The system shall allow authorized administrators to view system management information.

REQ-52: The system shall generate reports related to users, vehicles, bookings, services, technicians, and inventory.

REQ-53: The system shall allow reports to be filtered using supported parameters such as date, status, role, or category.

REQ-54: The system shall restrict administrative reports to authorized users.

REQ-55: The system shall display an appropriate message when insufficient data is available for a requested report.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- *The system shall process user login requests and display the appropriate dashboard within 3 seconds under normal operating conditions.*
- *Vehicle and service provider search functions shall display matching records within 3 seconds under normal database loads.*
- *The system shall update booking and service status information without unnecessary delay.*
- *Standard administrative reports shall be generated within 10 seconds under normal operating conditions.*
- *The system shall support multiple users accessing the application simultaneously without significant performance degradation.*

5.2 Safety Requirements

- *The system shall validate all user inputs before storing or processing them to prevent invalid vehicle, booking, service, and inventory records.*
- *Confirmation messages shall be displayed before permanently deleting important records.*
- *The system shall prevent invalid service bookings from being created.*
- *The system shall prevent spare parts from being issued when the required stock is unavailable.*
- *The system shall maintain consistency between service and inventory records.*
- *The system shall prevent unauthorized modifications to important service records.*

5.3 Security Requirements

- *The system shall use secure authentication mechanisms to verify users before providing access to protected functions.*
- *The system shall enforce role-based access for Administrator, Customer, Service Provider, Technician, and Inventory Manager accounts.*
- *Supabase Authentication shall be used to securely manage authentication and sessions where implemented.*
- *Row-Level Security shall be implemented to restrict access to database records according to user roles where applicable.*
- *The application shall validate and sanitize user inputs to reduce the risk of common attacks such as SQL Injection and Cross-Site Scripting.*
- *Administrative and other restricted functions shall only be accessible to authorized users.*
- *HTTPS shall be used for communication between users and the deployed application.*
- *Staff accounts shall not become active until the required Administrator approval is completed.*

5.4 Software Quality Attributes

- **Usability:** *The system shall provide simple, consistent, responsive, and role-specific interfaces so that users can perform common operations with minimal navigation.*
- **Maintainability:** *The application shall use modular and reusable React components and organized code to simplify debugging, maintenance, and future modifications.*
- **Reliability:** *The system shall maintain accurate customer, vehicle, booking, service, technician, and inventory records during normal operation.*
- **Scalability:** *The system shall support increasing numbers of users, vehicles, bookings, services, and inventory records without major changes to the application architecture.*
- **Portability:** *The web application shall be accessible through modern web browsers on desktops, laptops, tablets, and smartphones.*
- **Testability:** *Individual modules such as authentication, vehicle management, booking, service management, inventory, and reporting shall be independently testable.*

5.5 Business Rules

- *Only authenticated users with appropriate roles shall be allowed to access protected system functions.*
- *Only authorized Administrators shall manage administrative functions.*
- *Customers shall only be allowed to view and manage their permitted vehicle information.*
- *Customers shall be allowed to create service booking requests for registered vehicles.*
- *Service Providers shall be allowed to accept or reject booking requests.*
- *Only authorized Service Providers shall assign technicians.*
- *Technicians shall record vehicle inspection and service information.*
- *Technicians shall request spare parts when required for service.*
- *Inventory Managers shall manage spare-part stock and issue available parts.*
- *Inventory quantities shall be updated after spare parts are issued.*
- *Service requests shall progress through the defined service workflow.*
- *Quality checking shall be performed before final service completion.*
- *Customers shall only be allowed to view their permitted service history.*
- *Staff roles requiring approval shall only become active after Administrator approval.*

6. Other Requirements

The application components—specifically the authentication module, role-based access control, user management, vehicle management, service provider selection, booking components, technician assignment, vehicle inspection, service management, inventory management, notification components, service history, and report generation modules—shall be designed using modular and reusable components.

The application shall be structured to facilitate easy maintenance and future integration with backend services such as a Spring Boot REST API.

The system shall maintain clear separation between user interface components, application logic, and data/service access to support future modifications.

Appendix A: Glossary

- **Administrator:** An authorized user with higher-level access to manage users, staff approvals, system records, reports, and administrative functions.
- **Customer:** An authorized user who can register vehicles, select service providers, book services, track service progress, and view service history.
- **Service Provider:** An authorized user responsible for managing service bookings, accepting or rejecting requests, assigning technicians, and monitoring service activities.
- **Technician:** An authorized service staff member responsible for vehicle inspection, service activities, work recording, parts requests, and progress updates.
- **Inventory Manager:** An authorized user responsible for managing spare parts, stock quantities, suppliers, low-stock items, and parts issuing.
- **Vehicle Record:** A system record containing information about a customer's registered vehicle.
- **Service Booking:** A request submitted by a customer containing vehicle, service provider, appointment, and service information.
- **Service Request:** A service activity created from an accepted booking and processed through the service workflow.
- **Vehicle Inspection:** The process of checking the vehicle and recording observations and service-related information.
- **Technician Assignment:** The process of assigning an available technician to an accepted service request.
- **Parts Request:** A request created by a technician for spare parts required to perform vehicle service.
- **Inventory Record:** A record containing information about spare parts, stock quantities, suppliers, and availability.
- **Service History:** A collection of completed service records associated with a vehicle.
- **Notification:** A system message provided to users regarding bookings, service progress, reminders, or other relevant events.
- **Role-Based Access Control (RBAC):** A security mechanism that provides system permissions based on the user's assigned role.
- **Supabase:** A backend platform that can provide PostgreSQL database services, authentication, Row-Level Security, and related services.
- **Row-Level Security (RLS):** A database security mechanism used to control which records users can access or modify.
- **React.js:** A JavaScript library used to develop the user interface of the system.
- **Vite:** A frontend development and build tool used for the React application.

Appendix B: Analysis Models

1. UML Activity Diagram

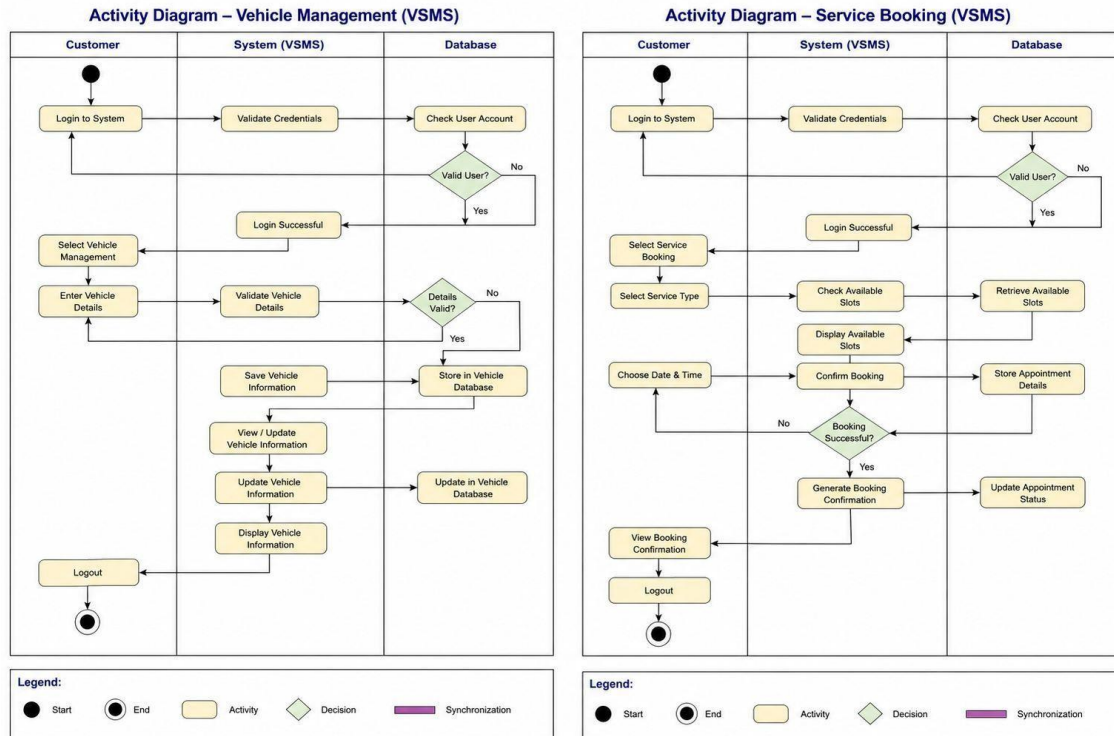


Figure B.1: UML Activity Diagram – Vehicle Service Management Workflow

2. UML Sequence Diagram

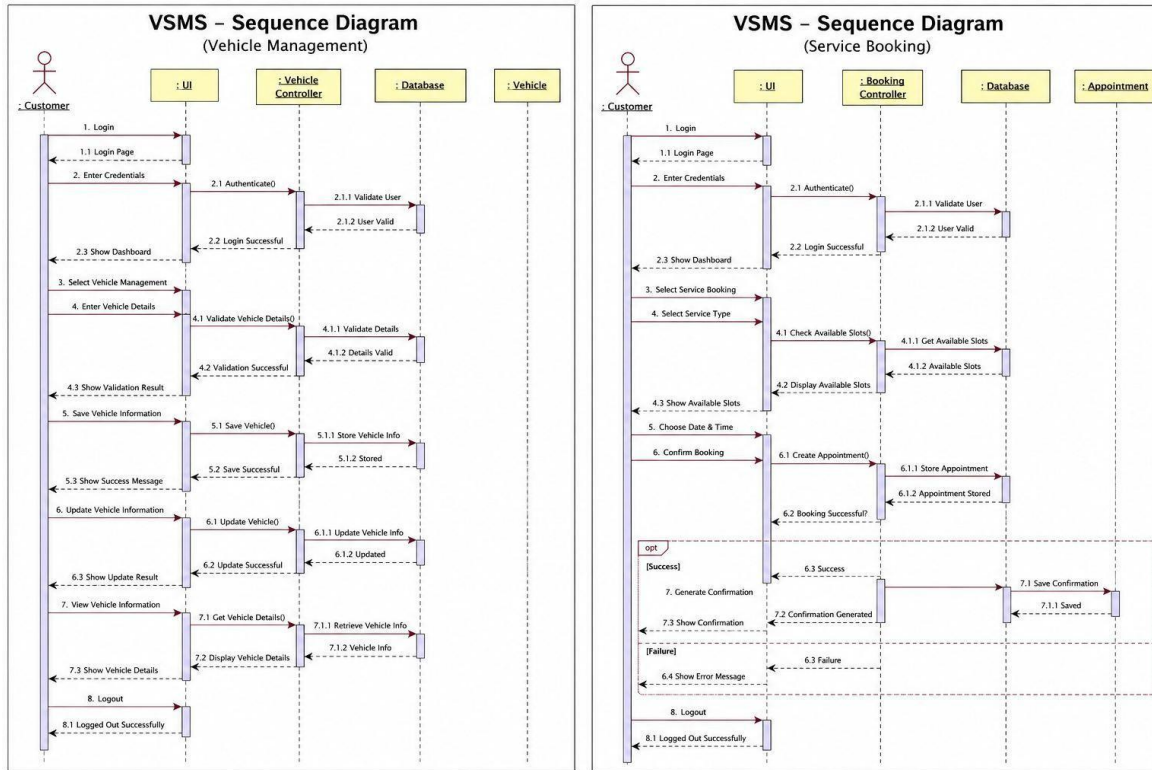


Figure B.2: UML Sequence Diagram – Service Booking and Approval

Figure B.3: UML Sequence Diagram – Vehicle Service and Parts Request

3. State Chart Diagram

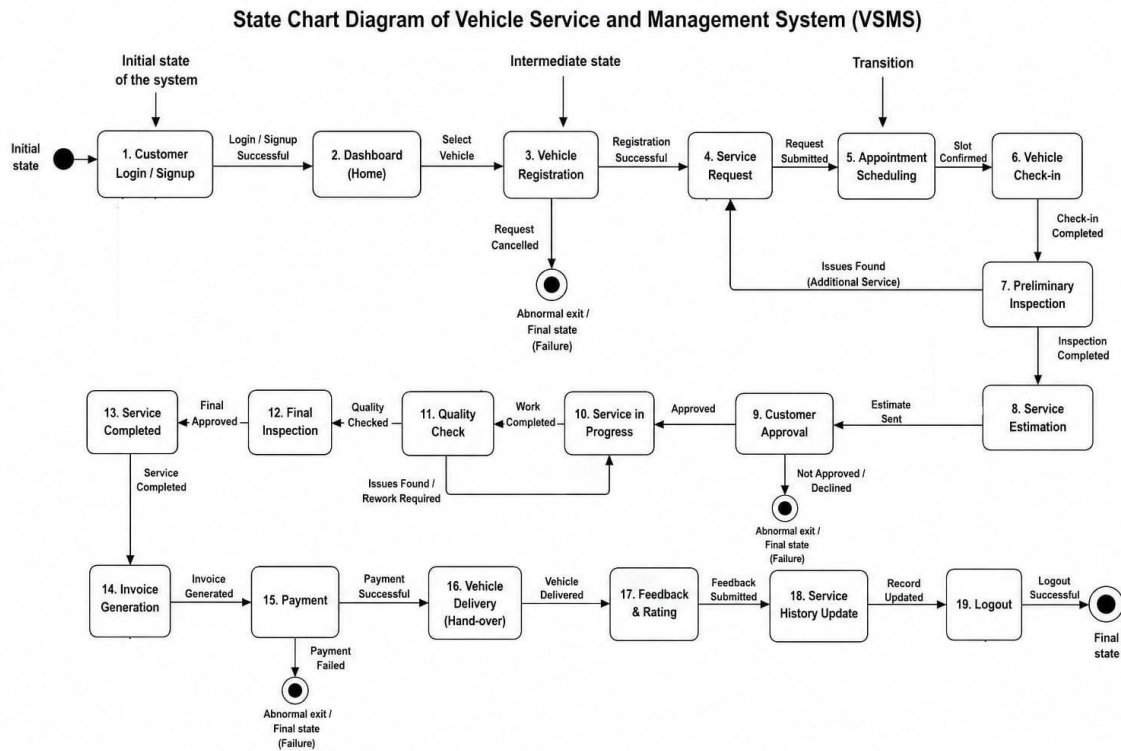


Figure B.4: UML State Chart Diagram – Vehicle Service Request Lifecycle

4. UML Class Diagram

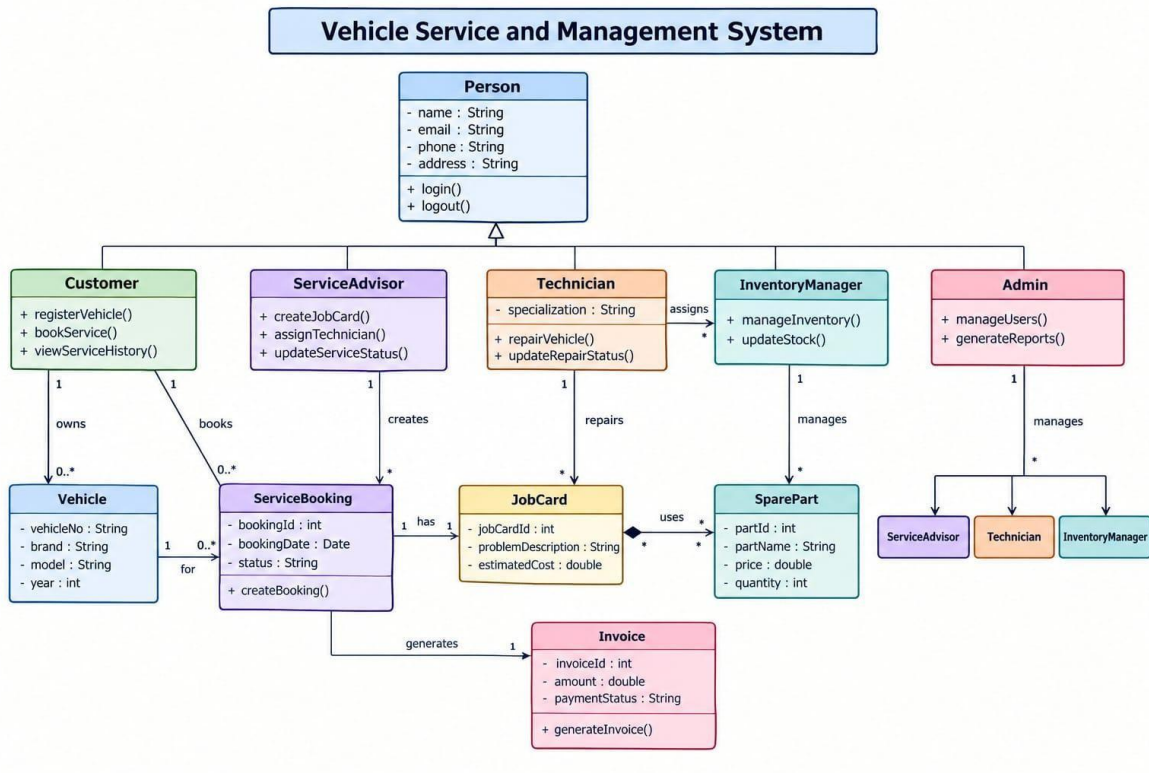


Figure B.5: UML Class Diagram – Smart Vehicle Service & Management System

5. UML Component Diagram

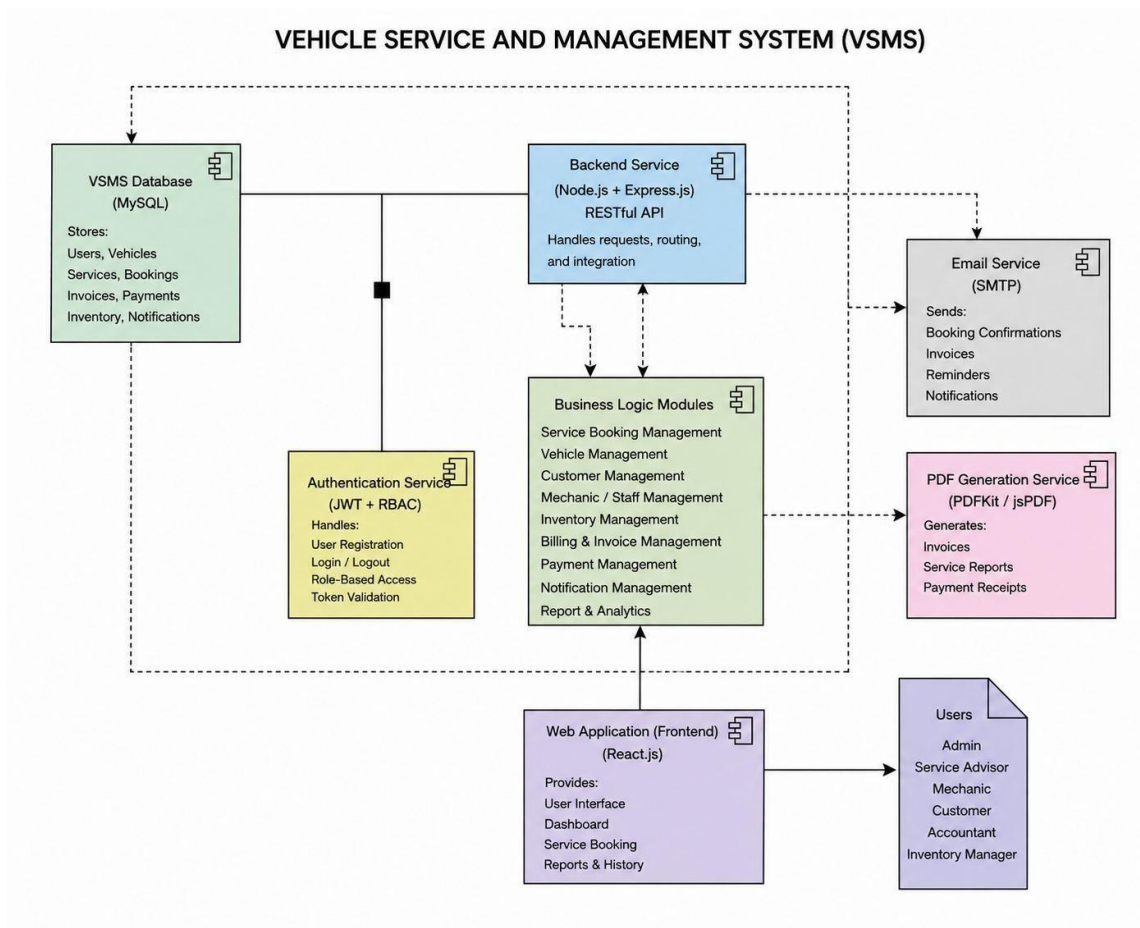


Figure B.6: UML Component Diagram – Smart Vehicle Service & Management System

6. UML Deployment Diagram

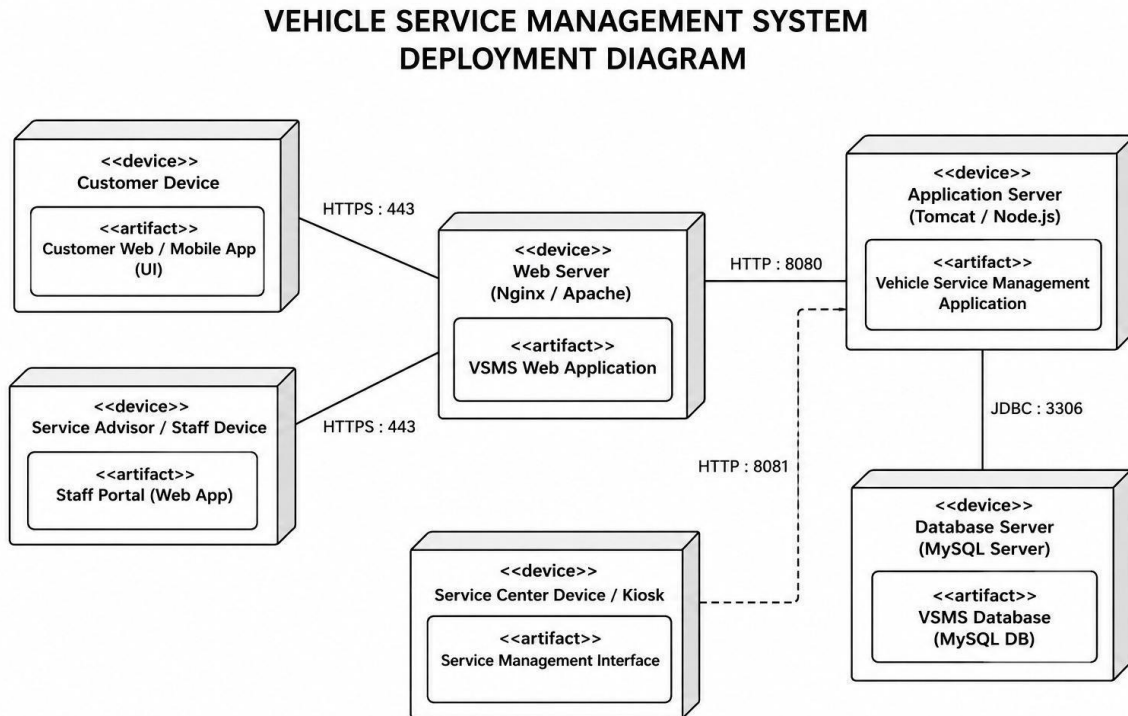


Figure B.7: UML Deployment Diagram – Smart Vehicle Service & Management System

Source Code

1. src/pages/customer/MyVehicles.jsx — Customer Vehicle Garage

```
"use client";

import React, { useState } from 'react';
import { Link } from 'react-router-dom';
import { Car, Plus, Edit2, Trash2, Calendar, Shield, Award, Gauge } from 'lucide-react';
import { useAuth } from '../context/AuthContext';
import { useData } from '../context/DataContext';
import { PageHeader } from
'../components/common/PageHeader';
import { AddEditVehicleModal } from
'./AddEditVehicleModal';
import { ConfirmationDialog } from
'../components/common/ConfirmationDialog';
import { formatDate } from '../utils/formatters';

export const MyVehicles = () => {
  const { currentUser } = useAuth();
  const { vehicles, updateVehicle,
deleteVehicle } = useData();

  const [isAddModalOpen, setIsAddModalOpen] =
useState(false);
  const [editingVehicle, setEditingVehicle] =
useState(null);
  const [deletingVehicleId, setDeletingVehicleId] =
useState(null);

  const myVehicles = vehicles.filter((v) => v.ownerId
=== currentUser?.id || !v.ownerId);

  const handleSaveVehicle = (vehicleData) => {
    if (editingVehicle) {
      updateVehicle(editingVehicle.id, vehicleData);
    } else {
      addVehicle(vehicleData);
    }
    setIsAddModalOpen(false);
    setEditingVehicle(null);
  };

  const handleConfirmDelete = () => {
    if (deletingVehicleId) {
      deleteVehicle(deletingVehicleId);
      setDeletingVehicleId(null);
    }
  };

  return (
    <div className="space-y-6">
      <PageHeader
        title="My Digital Garage"
        subtitle="Manage your registered vehicles and
compliance status"
        breadcrumbs={[{ label: 'My Vehicles' }]}
        actions={
          <button
            onClick={() => { setEditingVehicle(null);
setIsAddModalOpen(true); }}
            className="btn btn-primary flex items-
center gap-1.5"
            >
            <Plus size={16} /> Add New Vehicle
          </button>
        >
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2
lg:grid-cols-3 gap-6">
        {myVehicles.map((vehicle) => {
          <div key={vehicle.id} className="card bg-
white rounded-lg border border-slate-200 overflow-
hidden shadow-sm">
            <div className="relative h-44">
              <img src={vehicle.image}
alt={vehicle.model} className="w-full h-full object-
cover" />
              <span className="absolute top-3 left-3
bg-slate-900/80 text-white font-mono text-xs px-2.5 py-
1 rounded">
```

```
              {vehicle.registrationNumber}
            </span>
            <span className="absolute top-3 right-3 bg-
white/90 text-slate-800 text-xs px-2 py-0.5 rounded font-
bold">
              {vehicle.manufacturingYear} &bull;
            </span>
            </div>

            <div className="p-4 space-y-3">
              <div className="flex justify-between items-
center">
                <h3 className="text-base font-bold text-slate-
900">{vehicle.brand} {vehicle.model}</h3>
                <span className="text-xs bg-blue-50 text-blue-
700 px-2 py-0.5 rounded font-
semibold">{vehicle.fuelType}</span>
              </div>

              <div className="grid grid-cols-2 gap-2 text-xs
bg-slate-50 p-3 rounded">
                <div><span className="text-slate-
500">Odometer:</span> <strong>{vehicle.mileage}</strong></div>
                <div><span className="text-slate-500">Last
Service:</span>
                <strong>{formatDate(vehicle.lastServiceDate)}</strong></div>
                <div><span className="text-slate-
500">Insurance Till:</span>
                <strong>{formatDate(vehicle.insuranceExpiry)}</strong></div>
                <div><span className="text-slate-500">PUC
Expiry:</span>
                <strong>{formatDate(vehicle.pucExpiry)}</strong></div>
              </div>

              <div className="flex justify-between items-
center pt-2 border-t border-slate-100">
                <div className="flex gap-2">
                  <button onClick={() => {
setEditingVehicle(vehicle); setIsAddModalOpen(true); }}
                    className="btn btn-secondary btn-sm">
                    <Edit2 size={13} /> Edit
                  </button>
                  <button onClick={() =>
setDeletingVehicleId(vehicle.id)} className="btn btn-danger
btn-sm">
                    <Trash2 size={13} />
                  </button>
                </div>
                <Link
to={`\customer/book?vehicleId=${vehicle.id}`} className="btn
btn-primary btn-sm">
                  Book Service & rarr;
                </Link>
              </div>
            </div>
          </div>
        </div>
      </div>

      <AddEditVehicleModal
        isOpen={isAddModalOpen}
        onClose={() => { setIsAddModalOpen(false);
setEditingVehicle(null); }}
        onSave={handleSaveVehicle}
        initialData={editingVehicle}
        ownerId={currentUser?.id}
      />

      <ConfirmationDialog
        isOpen={!deletingVehicleId}
        onClose={() => setDeletingVehicleId(null)}
        onConfirm={handleConfirmDelete}
        title="Delete Vehicle"
        message="Are you sure you want to remove this vehicle
from your garage?"
      />
    </div>
  );
};
```

2. src/pages/customer/BookService.jsx — Service Appointment Booking

```

"use client";

import React, { useState } from 'react';
import { useNavigate, useSearchParams } from 'react-router-dom';
import { Calendar, Clock, Car, Wrench, AlertCircle } from 'lucide-react';
import { useAuth } from '../context/AuthContext';
import { useData } from '../context/DataContext';
import { SERVICE_TYPES } from '../data';
import { PageHeader } from '../components/common/PageHeader';

export const BookService = () => {
  const { currentUser } = useAuth();
  const { vehicles, providers, bookAppointment } = useData();
  const navigate = useNavigate();
  const [searchParams] = useSearchParams();

  const myVehicles = vehicles.filter((v) => v.ownerId === currentUser?.id || !v.ownerId);
  const nexaCareCenter = providers[0] || { id: 'usr-prov-1', name: 'NexaCare Service Center' };

  const [vehicleId, setVehicleId] = useState(searchParams.get('vehicleId') || (myVehicles[0]?.id || ''));
  const [serviceType, setServiceType] = useState('General Service');
  const [scheduledDate, setScheduledDate] = useState(() => {
    const tomorrow = new Date();
    tomorrow.setDate(tomorrow.getDate() + 1);
    return tomorrow.toISOString().split('T')[0];
  });
  const [scheduledTime, setScheduledTime] = useState('10:00 AM');
  const [problemDescription, setProblemDescription] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const selectedVehicle = vehicles.find((v) => v.id === vehicleId);

  const estimatedPrices = {
    'General Service': '₹1,800',
    'Oil Change': '₹750',
    'Brake Service': '₹1,450',
    'Engine Service': '₹2,200',
    'Periodic Maintenance': '₹1,600'
  };

  const handleBook = async (e) => {
    e.preventDefault();
    if (!vehicleId || !serviceType || !scheduledDate || !scheduledTime) {
      setError('Please fill in all mandatory booking fields.');
```

```

        <textarea
          rows={3}
          value={problemDescription}
          onChange={ (e) =>
            setProblemDescription(e.target.value)}
          placeholder="Describe complaints or
            routine checklist..."
          className="w-full px-3 py-2 border
            rounded text-xs"
          required
        />
      </div>
    </div>
  </div>

  <div className="bg-white p-5 rounded-lg border
    border-slate-200 h-fit space-y-4">
    <h3 className="font-bold text-sm text-slate-
      900 border-b pb-2">Booking Summary</h3>
    <div className="space-y-2 text-xs text-slate-
      600">
      <div className="flex justify-
        between"><span>Workshop:</span>
      <strong>{nexaCareCenter.name}</strong></div>
      <div className="flex justify-
        between"><span>Package:</span>
      <strong>{serviceType}</strong></div>
      <div className="flex justify-
        between"><span>Date:</span> <strong>{scheduledDate}
        ({scheduledTime})</strong></div>
      <div className="flex justify-between
        border-t pt-2 text-sm font-bold text-slate-900">
        <span>Estimated Cost:</span>
        <span className="text-emerald-
          600">{estimatedPrices[serviceType] || '₹1,500'}</span>
      </div>
    </div>
    <button type="submit" disabled={loading}
      className="w-full btn btn-primary py-2 text-xs font-
        bold">
      {loading ? 'Scheduling...' : 'Confirm
        Booking'}
    </button>
  </div>
</form>
</div>
);
};

```

3. src/pages/customer/ServiceTracking.jsx — Live Service Tracking

```

"use client";

import React from 'react';
import { Link } from 'react-router-dom';
import { Activity } from 'lucide-react';
import { useAuth } from '../../../context/AuthContext';
import { useData } from '../../../context/DataContext';
import { PageHeader } from
  '../../../components/common/PageHeader';
import { StatusBadge } from
  '../../../components/common/StatusBadge';
import { ServiceTimeline } from
  '../../../components/common/ServiceTimeline';
import { formatDate, formatCurrency } from
  '../../../utils/formatters';

export const ServiceTracking = () => {
  const { currentUser } = useAuth();
  const { appointments } = useData();

  const myAppointments = appointments.filter(
    (a) => a.customerId === currentUser?.id ||
    a.customerName === currentUser?.name
  );

  const activeServices = myAppointments.filter(
    (a) => ![ 'COMPLETED', 'VEHICLE_READY', 'REJECTED',
    'CANCELLED' ].includes(a.status)
  );

  return (
    <div className="space-y-6">
      <PageHeader
        title="Live Vehicle Service Tracking"

```

```

        subtitle="Real-time progression from arrival to
        vehicle handover"
      />

      {activeServices.length > 0 ? (
        <div className="space-y-5">
          {activeServices.map((service) => (
            <div key={service.id} className="bg-white p-6
              rounded-lg border-l-4 border-blue-600 border-shadow-sm space-
                y-4">
              <div className="flex justify-between items-
                center">
                <div>
                  <div className="flex items-center gap-2 mb-
                    1">
                    <span className="text-xs font-mono font-
                      bold text-blue-600">JOB #{service.id}</span>
                    <StatusBadge status={service.status} />
                  </div>
                  <h3 className="text-lg font-bold text-slate-
                    900">{service.vehicleName}</h3>
                  <p className="text-xs text-slate-
                    500">Package: {service.serviceType} &bull; Center:
                    {service.providerName}</p>
                  </div>
                  <Link
                    to={` /customer/appointments/${service.id}`} className="btn
                    btn-primary btn-sm">
                    View Details &rarr;
                  </Link>
                </div>
                <ServiceTimeline currentStatus={service.status}
              />
            </div>
          )})
        </div>
      ) : (
        <div className="text-center p-12 bg-white rounded-lg
          border border-slate-200 text-slate-400">
          <Activity className="mx-auto mb-2 opacity-50"
            size={36} />
          <p>No active services in the bay currently.</p>
        </div>
      )
    </div>
  );
};

```

4. src/pages/customer/RemindersPage.jsx — Reminders and Compliance Alerts

```

"use client";

import React, { useState } from 'react';
import { Link } from 'react-router-dom';
import { Clock, Shield, Award, Calendar, Wrench } from
  'lucide-react';
import { useAuth } from '../../../context/AuthContext';
import { useData } from '../../../context/DataContext';
import { PageHeader } from
  '../../../components/common/PageHeader';
import { formatDate } from '../../../utils/formatters';

const calcDaysRemaining = (dueDate) => {
  if (!dueDate) return null;
  const today = new Date();
  today.setHours(0, 0, 0, 0);
  const due = new Date(dueDate);
  due.setHours(0, 0, 0, 0);
  return Math.round((due - today) / (1000 * 60 * 60 * 24));
};

```

```

const calcUrgency = (days) => {
  if (days === null) return 'LOW';
  if (days <= 0) return 'OVERDUE';
  if (days <= 30) return 'HIGH';
  if (days <= 60) return 'MEDIUM';
  return 'LOW';
};

export const RemindersPage = () => {
  const { currentUser } = useAuth();
  const { reminders } = useData();
  const [filterType, setFilterType] = useState('ALL');

  const myReminders = (reminders || []).
    .filter((r) => !r.userId || r.userId ===
      currentUser?.id || r.userId === 'usr-cust-1' ||
      !currentUser)
    .map((r) => {
      const days = calcDaysRemaining(r.dueDate);
      const urgency = calcUrgency(days);
      return { ...r, daysRemaining: days, urgency };
    })
    .sort((a, b) => a.daysRemaining - b.daysRemaining);

  const filteredReminders = myReminders.filter((r) =>
    filterType === 'ALL' || r.type === filterType);

  return (
    <div className="space-y-6">
      <PageHeader
        title="Vehicle Reminders & Compliance"
        subtitle="Automated alerts for periodic
service, insurance, and PUC renewals"
      />

      <div className="flex gap-3 flex-wrap">
        {[ 'ALL', 'NEXT_SERVICE', 'INSURANCE_EXPIRY',
'PUC_EXPIRY' ].map((type) => (
          <button
            key={type}
            onClick={() => setFilterType(type)}
            className={`px-3 py-1.5 rounded text-xs
font-semibold ${
              filterType === type ? 'bg-blue-600 text-
white' : 'bg-white border text-slate-700'
            }`}
          >
            {type.replace('_', ' ')}
          </button>
        ))}
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2
lg:grid-cols-3 gap-6">
        {filteredReminders.map((rem) => (
          <div key={rem.id} className="bg-white p-5
rounded-lg border border-slate-200 shadow-sm space-y-
3">
            <div className="flex justify-between items-
start">
              <h4 className="font-bold text-sm text-
slate-900">{rem.title}</h4>
              <span className={`text-[10px] font-bold
px-2 py-0.5 rounded ${
                rem.urgency === 'OVERDUE' ? 'bg-red-50
text-red-600' :
                rem.urgency === 'HIGH' ? 'bg-amber-50
text-amber-700' : 'bg-blue-50 text-blue-600'
              }`}>
                {rem.urgency}
              </span>
            </div>
            <p className="text-xs text-slate-
600">{rem.description}</p>
            <div className="flex justify-between items-
center text-xs pt-3 border-t">
              <span className="text-slate-500">Due:
<strong>{formatDate(rem.dueDate)}</strong></span>
              <Link to="/customer/book" className="btn
btn-primary btn-sm">
                Book Now & rarr;
              </Link>
            </div>
          </div>
        ))}
      </div>

```

```

</div>
);
};

```

5. src/pages/provider/ProviderDashboard.jsx — Provider Workshop Dashboard

```

"use client";

import React, { useState } from 'react';
import { Link } from 'react-router-dom';
import { ClipboardList, Activity, HardHat, CheckCircle2,
Check, X, UserCheck } from 'lucide-react';
import { useData } from '../context/DataContext';
import { PageHeader } from
'../components/common/PageHeader';
import { StatusBadge } from
'../components/common/StatusBadge';
import { Modal } from '../components/common/Modal';

export const ProviderDashboard = () => {
  const { appointments, technicians, updateAppointmentStatus,
assignTechnician } = useData();
  const [selectedAptForAssign, setSelectedAptForAssign] =
useState(null);
  const [selectedTechId, setSelectedTechId] = useState('');

  const pendingRequests = appointments.filter((a) => a.status
=== 'PENDING');
  const activeBayServices = appointments.filter(
(a) => ![ 'COMPLETED', 'VEHICLE_READY', 'REJECTED',
'CANCELLED' ].includes(a.status)
);
  const completedServices = appointments.filter((a) =>
a.status === 'COMPLETED');

  const handleAccept = (aptId) =>
updateAppointmentStatus(aptId, 'ACCEPTED');
  const handleReject = (aptId) =>
updateAppointmentStatus(aptId, 'REJECTED');

  const handleAssign = () => {
    if (selectedAptForAssign && selectedTechId) {
      assignTechnician(selectedAptForAssign.id,
selectedTechId);
      setSelectedAptForAssign(null);
      setSelectedTechId('');
    }
  };

  return (
    <div className="space-y-6">
      <PageHeader title="Workshop Management Dashboard"
subtitle="Operational view of service bay workload" />

      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-
cols-4 gap-4">
        <div className="bg-white p-4 rounded-lg border shadow-
sm">
          <p className="text-2xl font-bold text-amber-
600">{pendingRequests.length}</p>
          <p className="text-xs text-slate-500">Pending
Requests</p>
        </div>
        <div className="bg-white p-4 rounded-lg border shadow-
sm">
          <p className="text-2xl font-bold text-blue-
600">{activeBayServices.length}</p>
          <p className="text-xs text-slate-500">Active in
Bays</p>
        </div>
        <div className="bg-white p-4 rounded-lg border shadow-
sm">
          <p className="text-2xl font-bold text-purple-
600">{technicians.length}</p>
          <p className="text-xs text-slate-
500">Technicians</p>
        </div>
        <div className="bg-white p-4 rounded-lg border shadow-
sm">
          <p className="text-2xl font-bold text-emerald-
600">{completedServices.length}</p>
          <p className="text-xs text-slate-500">Completed
Orders</p>
        </div>
      </div>

```

```

    <div className="bg-white rounded-lg border
shadow-sm p-4 space-y-3">
    <h3 className="font-bold text-sm text-slate-
900">Incoming Service Bookings</h3>
    {pendingRequests.map((req) => (
    <div key={req.id} className="flex justify-
between items-center p-3 border rounded text-xs">
    <div>
    <span className="font-mono font-bold
text-blue-600">#{req.id}</span> -
<strong>{req.vehicleName}</strong>
    <div className="text-slate-500">Package:
{req.serviceType} &bull; Customer:
{req.customerName}</div>
    </div>
    <div className="flex gap-2">
    <button onClick={() =>
handleAccept(req.id)} className="btn btn-success btn-
sm">Accept</button>
    <button onClick={() => {
setSelectedAptForAssign(req);
setSelectedTechId(technicians[0]?.id || ''); }}
className="btn btn-primary btn-sm">
    Assign Tech
    </button>
    <button onClick={() =>
handleReject(req.id)} className="btn btn-danger btn-
sm">Decline</button>
    </div>
    </div>
    ))}
    </div>

    {selectedAptForAssign && (
    <Modal
    isOpen={!!selectedAptForAssign}
    onClose={() => setSelectedAptForAssign(null)}
    title="Assign Technician"
    >
    <div className="space-y-3 text-xs">
    <p>Assign mechanic for
<strong>{selectedAptForAssign.vehicleName}</strong></p>
    <select value={selectedTechId}
onChange={(e) => setSelectedTechId(e.target.value)}
className="w-full p-2 border rounded">
    {technicians.map((t) => (
    <option key={t.id}
value={t.id}>{t.name} ({t.specialty})</option>
    ))}
    </select>
    <button onClick={handleAssign}
className="w-full btn btn-primary py-1.5">Confirm
Assignment</button>
    </div>
    </Modal>
    ))}
    </div>
  );
};

```

6. src/pages/customer/ServiceHistory.jsx — Service History and Receipts

"use client";

```

import React, { useState } from 'react';
import { History, FileText, Download } from 'lucide-
react';
import { useAuth } from '../../../context/AuthContext';
import { useData } from '../../../context/DataContext';
import { PageHeader } from
'../../../components/common/PageHeader';
import { StatusBadge } from
'../../../components/common/StatusBadge';
import { SearchBar } from
'../../../components/common/SearchBar';
import { DataTable } from
'../../../components/common/DataTable';
import { Modal } from '../../../components/common/Modal';
import { formatDate, formatCurrency } from
'../../../utils/formatters';

export const ServiceHistory = () => {
  const { currentUser } = useAuth();
  const { appointments } = useData();

```

```

  const [searchQuery, setSearchQuery] = useState('');
  const [selectedInvoice, setSelectedInvoice] =
useState(null);

  const historyRecords = (appointments || []).filter(
    (a) =>
    (a.customerId === currentUser?.id || a.customerName ===
currentUser?.name) &&
    ['COMPLETED', 'VEHICLE_READY', 'CANCELLED',
'REJECTED'].includes(a.status)
  );

  const filteredRecords = historyRecords.filter((r) => {
    const q = searchQuery.toLowerCase();
    return (
    r.id.toLowerCase().includes(q) ||
    r.vehicleName.toLowerCase().includes(q) ||
    r.serviceType.toLowerCase().includes(q) ||
    r.providerName.toLowerCase().includes(q)
    );
  });

  const columns = [
    { header: 'Job ID', accessor: 'id', render: (row) => <span
className="font-mono font-bold text-blue-600">{row.id}</span>
},
    {
      header: 'Vehicle',
      accessor: 'vehicleName',
      render: (row) => (
        <div>
        <div className="font-bold">{row.vehicleName}</div>
        <div className="text-xs text-slate-
500">{row.registrationNumber}</div>
        </div>
      )
    },
    { header: 'Service Package', accessor: 'serviceType' },
    { header: 'Completed Date', accessor: 'scheduledDate',
render: (row) => formatDate(row.scheduledDate) },
    { header: 'Status', accessor: 'status', render: (row) =>
<StatusBadge status={row.status} /> },
    { header: 'Total Paid', accessor: 'estimatedCost', render:
(row) => <span className="font-bold text-emerald-
600">{formatCurrency(row.estimatedCost)}</span> },
    {
      header: 'Invoice',
      align: 'right',
      render: (row) => (
        <button onClick={() => setSelectedInvoice(row)}
className="btn btn-secondary btn-sm flex items-center gap-1">
        <FileText size={14} /> Receipt
        </button>
      )
    }
  ];

  return (
    <div className="space-y-6">
    <PageHeader
    title="Vehicle Service History"
    subtitle="Digital logbook of completed maintenance,
parts replacements, and receipts"
    />

    <div className="bg-white p-4 rounded-lg border border-
slate-200">
    <SearchBar value={searchQuery}
onChange={setSearchQuery} placeholder="Search service history
by ID, vehicle, or workshop..." />
    </div>

    <div className="bg-white rounded-lg border border-slate-
200 overflow-hidden">
    <DataTable columns={columns} data={filteredRecords}
pageSize={8} />
    </div>

    {selectedInvoice && (
    <Modal
    isOpen={!!selectedInvoice}
    onClose={() => setSelectedInvoice(null)}
    title="Digital Service Summary & Receipt"
    subtitle={`Reference Job ID: ${selectedInvoice.id}`}
    >
    <div className="space-y-4 text-xs">

```

```


## 7. src/pages/technician/AssignedServices.jsx — Technician Assigned Services Queue



"use client";



```

import React, { useState } from 'react';
import { Link } from 'react-router-dom';
import { Wrench, ShieldCheck, Play } from 'lucide-
react';
import { useAuth } from '../context/AuthContext';
import { useData } from '../context/DataContext';
import { PageHeader } from
'../components/common/PageHeader';
import { StatusBadge } from
'../components/common/StatusBadge';
import { SearchBar } from
'../components/common/SearchBar';
import { DataTable } from
'../components/common/DataTable';
import { formatDate } from '../utils/formatters';

export const AssignedServices = () => {
 const { currentUser } = useAuth();
 const { appointments, updateAppointmentStatus } =
useData();
 const [searchQuery, setSearchQuery] = useState('');

 const myAssignedJobs = (appointments || []).filter(
(a) =>
(a.technicianId === currentUser?.id ||
a.technicianName?.includes('Vignesh') ||
!a.technicianId) &&
!['COMPLETED', 'VEHICLE READY', 'CANCELLED',

```



```

'REJECTED'].includes(a.status)
);

const filtered = myAssignedJobs.filter((j) => {
 const q = searchQuery.toLowerCase();
 return (
 j.id.toLowerCase().includes(q) ||
 j.vehicleName.toLowerCase().includes(q) ||
 j.serviceType.toLowerCase().includes(q)
);
});

const handleAdvance = (job) => {
 const workflow = [
 'TECHNICIAN_ASSIGNED',
 'VEHICLE_RECEIVED',
 'INSPECTION',
 'IN_PROGRESS',
 'QUALITY_CHECK',
 'COMPLETED'
];
 const currentIndex = workflow.indexOf(job.status);
 if (currentIndex >= 0 && currentIndex < workflow.length -
1) {
 updateAppointmentStatus(job.id, workflow[currentIndex +
1]);
 }
};

const columns = [
 { header: 'Job ID', accessor: 'id', render: (row) => #{row.id}
},
 {
 header: 'Vehicle & Plate',
 accessor: 'vehicleName',
 render: (row) => (
 <div>
 <div className="font-bold">{row.vehicleName}</div>
 <div className="text-xs text-slate-
500">{row.registrationNumber}</div>
 </div>
),
 },
 { header: 'Service Package', accessor: 'serviceType' },
 { header: 'Customer', accessor: 'customerName' },
 { header: 'Scheduled Slot', accessor: 'scheduledDate',
render: (row) => ` ${formatDate(row.scheduledDate)}
(${row.scheduledTime})` },
 { header: 'Status', accessor: 'status', render: (row) =>
<StatusBadge status={row.status} /> },
 {
 header: 'Actions',
 align: 'right',
 render: (row) => (
 <div className="flex gap-2 justify-end">
 <Link to={`/technician/service/${row.id}`}
className="btn btn-secondary btn-sm flex items-center gap-1">
 <ShieldCheck size={14} /> Work Log
 </Link>
 <button onClick={() => handleAdvance(row)}
className="btn btn-primary btn-sm flex items-center gap-1">
 <Play size={13} /> Next Stage &arr;
 </button>
 </div>
)
 }
];

return (
 <div className="space-y-6">
 <PageHeader
 title="Assigned Technical Services Queue"
 subtitle="Manage assigned vehicle diagnostics,
inspection stages, and repair progression"
 />

 <div className="bg-white p-4 rounded-lg border border-
slate-200">
 <SearchBar value={searchQuery}
onChange={setSearchQuery} placeholder="Search assigned work
orders by vehicle or ID..." />
 </div>

 <div className="bg-white rounded-lg border border-slate-
200 overflow-hidden">

```


```

```

        <DataTable columns={columns} data={filtered}
pageSize={8} />
      </div>
    </div>
  );
};

```

8. src/pages/inventory/StockManagement.jsx — Stock Level Management

```

"use client";

import React, { useState } from 'react';
import { Plus, Minus, CheckCircle2 } from 'lucide-react';
import { useData } from '../../context/DataContext';
import { PageHeader } from '../../components/common/PageHeader';
import { SearchBar } from '../../components/common/SearchBar';
import { formatCurrency } from '../../utils/formatters';

export const StockManagement = () => {
  const { spareParts, adjustStock } = useData();
  const [searchQuery, setSearchQuery] = useState('');
  const [feedback, setFeedback] = useState('');

  const filteredParts = (spareParts || []).filter((p) => {
    const pName = (p.partName || p.name || '').toLowerCase();
    const pNum = (p.partNumber || p.part_number || '').toLowerCase();
    const q = searchQuery.toLowerCase();
    return pName.includes(q) || pNum.includes(q);
  });

  const handleQuickAdjust = (partId, delta) => {
    adjustStock(partId, delta);
    setFeedback(`Stock adjusted by ${delta > 0 ? '+' : ''} ${Math.abs(delta)} units.`);
    setTimeout(() => setFeedback(''), 3000);
  };

  return (
    <div className="space-y-6">
      <PageHeader
        title="Stock Level Operations & Inventory"
        subtitle="Manage stock intakes, warehouse shelf adjustments, and quantity counts"
      />

      {feedback && (
        <div className="p-3 bg-emerald-50 text-emerald-700 text-xs rounded border border-emerald-200 flex items-center gap-2">
          <CheckCircle2 size={16} />
          <span>{feedback}</span>
        </div>
      )}

      <div className="bg-white p-4 rounded-lg border border-slate-200">
        <SearchBar value={searchQuery}
          onChange={setSearchQuery} placeholder="Search parts by name or SKU to adjust quantity..." />
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
        {filteredParts.map((part) => {
          const qty = part.availableQuantity ?? 0;
          const min = part.minimumStock ?? 0;
          const isLow = qty <= min;

          return (
            <div key={part.id} className="bg-white p-5 rounded-lg border border-slate-200 shadow-sm flex flex-col justify-between space-y-4">
              <div>
                <div className="flex justify-between items-start">
                  <div>
                    <h3 className="font-bold text-sm text-slate-900">{part.partName || part.name}</h3>

```

```

                <p className="text-xs font-mono text-blue-600">{part.partNumber || part.part_number}</p>
              </div>
              <span className={`text-xl font-black ${isLow ? 'text-rose-600' : 'text-emerald-600'}`}>
                {qty}
              </span>
            </div>
            <div className="flex justify-between text-xs text-slate-500 mt-2">
              <span>Min Stock: {min}</span>
              <span>Shelf: {part.location || 'A-01'}</span>
              <span>{formatCurrency(part.unitPrice ?? 0)}/unit</span>
            </div>
          </div>
        </div>

        <div className="flex items-center justify-between pt-3 border-t">
          <span className="text-xs text-slate-500 font-semibold">Quick Adjustment:</span>
          <div className="flex gap-1.5">
            <button onClick={() => handleQuickAdjust(part.id, -1)} className="btn btn-secondary btn-sm px-2">
              <Minus size={12} /> -1
            </button>
            <button onClick={() => handleQuickAdjust(part.id, 1)} className="btn btn-secondary btn-sm px-2">
              <Plus size={12} /> +1
            </button>
            <button onClick={() => handleQuickAdjust(part.id, 5)} className="btn btn-primary btn-sm px-2.5">
              +5 Units
            </button>
          </div>
        </div>
      </div>
    );
  };
};

```

9. src/pages/admin/UserManagement.jsx — User and Staff Administration

```

"use client";

import React, { useState } from 'react';
import { Users, Plus, CheckCircle2, UserCheck, UserX } from 'lucide-react';
import { useData } from '../../context/DataContext';
import { useAuth } from '../../context/AuthContext';
import { PageHeader } from '../../components/common/PageHeader';
import { SearchBar } from '../../components/common/SearchBar';
import { Modal } from '../../components/common/Modal';

export const UserManagement = () => {
  const { users = [], updateUserStatus, createStaffMember } = useData();
  const { createStaffAccountByAdmin } = useAuth();

  const [searchQuery, setSearchQuery] = useState('');
  const [roleFilter, setRoleFilter] = useState('ALL');
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    phone: '',
    role: 'TECHNICIAN',
    specialty: 'Engine Diagnostics & Inspection',
    password: 'Password@123'
  });

  const filteredUsers = users.filter((u) => {
    const q = searchQuery.toLowerCase();
    const matchesSearch = u.name?.toLowerCase().includes(q) || u.email?.toLowerCase().includes(q);
    const matchesRole = roleFilter === 'ALL' || u.role === roleFilter;
  });

```

```

        {user.status || 'ACTIVE'}
      </span>

      <button
        onClick={() => handleToggleStatus(user)}
        className="btn btn-secondary btn-sm flex
items-center gap-1"
      >
        {user.status === 'ACTIVE' ? <UserX size={13}>
        /> : <UserCheck size={13}> />}
        {user.status === 'ACTIVE' ? 'Suspend' :
'Activate'}
      </button>
    </div>
  </div>
  )))
</div>

  {isModalOpen && (
    <Modal isOpen={isModalOpen} onClose={() =>
setIsModalOpen(false)} title="Provision New Staff Account">
      <form onSubmit={handleCreateStaff} className="space-
y-3 text-xs">
        <div>
          <label className="block font-semibold mb-1">Full
Name </label>
          <input required value={formData.name}
onChange={(e) => setFormData({ ...formData, name:
e.target.value })} className="w-full p-2 border rounded" />
        </div>
        <div>
          <label className="block font-semibold mb-
1">Email Address </label>
          <input type="email" required
value={formData.email} onChange={(e) => setFormData({
...formData, email: e.target.value })} className="w-full p-2
border rounded" />
        </div>
        <div>
          <label className="block font-semibold mb-1">Role
*</label>
          <select value={formData.role} onChange={(e) =>
setFormData({ ...formData, role: e.target.value })}
className="w-full p-2 border rounded">
            <option value="TECHNICIAN">Technician</option>
            <option value="SERVICE_PROVIDER">Service
Provider</option>
            <option value="INVENTORY_MANAGER">Inventory
Manager</option>
          </select>
        </div>
        <button type="submit" className="w-full btn btn-
primary py-2 font-bold">Create Staff Member</button>
      </form>
    </Modal>
  ))
</div>
);
};

```

```
"use client";
```

```
import React from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { Car, Calendar, Clock, Activity, CheckCircle2, Plus,
Building } from 'lucide-react';
import { useAuth } from '../../../context/AuthContext';
import { useData } from '../../../context/DataContext';
import { PageHeader } from
'../../../components/common/PageHeader';
import { DashboardCard } from
'../../../components/common/DashboardCard';
import { StatusBadge } from
'../../../components/common/StatusBadge';
import { formatDate } from '../../../utils/formatters';

export const CustomerDashboard = () => {
  const { currentUser } = useAuth();
  const { vehicles, appointments, reminders } = useData();
  const navigate = useNavigate();

  const myVehicles = vehicles.filter((v) => v.ownerId ===
currentUser?.id || !v.ownerId);
```

```

const myAppointments = appointments.filter((a) =>
a.customerId === currentUser?.id || a.customerName ===
currentUser?.name);
const myReminders = reminders.filter((r) => r.userId
=== currentUser?.id || !r.userId);

const activeServices = myAppointments.filter(
(a) => ![ 'COMPLETED', 'VEHICLE_READY', 'REJECTED',
'CANCELLED' ].includes(a.status)
);
const completedServices = myAppointments.filter(
(a) => a.status === 'COMPLETED' || a.status ===
'VEHICLE_READY'
);

return (
<div className="space-y-6">
  <PageHeader
    title={`Welcome, ${currentUser?.name} |
'Customer'}`}
    subtitle="NexaCare Customer Portal &bull;
Manage vehicles, track services, and schedule
maintenance"
    actions={
      <div className="flex gap-2">
        <Link to="/customer/book" className="btn
btn-primary flex items-center gap-1.5">
          <Plus size={16} /> Book Service
        </Link>
        <Link to="/customer/providers"
className="btn btn-secondary flex items-center gap-
1.5">
          <Building size={16} /> Service Center
        </Link>
      </div>
    }
  </div>

  <div className="grid grid-cols-1 sm:grid-cols-2
lg:grid-cols-4 gap-4">
    <DashboardCard
      title="My Vehicles"
      value={myVehicles.length}
      subtitle="Registered garage vehicles"
      icon={Car}
      color="primary"
      onClick={() =>
navigate('/customer/vehicles')}
    </DashboardCard>
    <DashboardCard
      title="Active Services"
      value={activeServices.length}
      subtitle="Currently in workshop bay"
      icon={Activity}
      color="warning"
      onClick={() =>
navigate('/customer/tracking')}
    </DashboardCard>
    <DashboardCard
      title="Completed Services"
      value={completedServices.length}
      subtitle="Past maintenance records"
      icon={CheckCircle2}
      color="success"
      onClick={() => navigate('/customer/history')}
    </DashboardCard>
    <DashboardCard
      title="Reminders"
      value={myReminders.length}
      subtitle="PUC & service due"
      icon={Clock}
      color="purple"
      onClick={() =>
navigate('/customer/reminders')}
    </div>

    {activeServices.length > 0 && (
      <div className="bg-white p-5 rounded-lg border-
1-4 border-blue-600 border shadow-sm flex flex-wrap
justify-between items-center gap-4">
        <div>
          <div className="flex items-center gap-2 mb-
1">
            <span className="text-xs font-bold text-
blue-600 uppercase">Ongoing Service</span>

```

11. src/pages/technician/TechnicianServiceDetails.jsx — Technician Inspection and Work Log

```
"use client";

import React, { useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { Wrench, ShieldCheck, CheckCircle2, Boxes, Plus, FileText } from 'lucide-react';
import { useData } from '../../context/DataContext';
import { useAuth } from '../../context/AuthContext';
import { PageHeader } from '../../components/common/PageHeader';
import { StatusBadge } from '../../components/common/StatusBadge';
import { ServiceTimeline } from '../../components/common/ServiceTimeline';
import { RequestSparePartsModal } from '../../RequestSparePartsModal';

export const TechnicianServiceDetails = () => {
  const { appointmentId } = useParams();
  const { appointments, partsRequests, updateAppointmentStatus } = useData();
  const { currentUser } = useAuth();
  const navigate = useNavigate();

  const appointment = appointments.find((a) => a.id === appointmentId) || appointments[0];
  const [isPartsModalOpen, setIsPartsModalOpen] = useState(false);
  const [feedback, setFeedback] = useState('');

  const [checklist, setChecklist] = useState({
    engineOil:
      appointment?.inspectionChecklist?.engineOil || 'Optimal',
    brakePads:
      appointment?.inspectionChecklist?.brakePads || 'Good (6mm)',
    batteryHealth:
      appointment?.inspectionChecklist?.batteryHealth || '95% Healthy',
    tyrePressure:
      appointment?.inspectionChecklist?.tyrePressure || '32 PSI',
    coolantLevel:
      appointment?.inspectionChecklist?.coolantLevel || 'Full',
    airFilter:
      appointment?.inspectionChecklist?.airFilter || 'Clean'
  });

  const [inspectionNotes, setInspectionNotes] = useState(
    appointment?.inspectionNotes || 'Completed multi-point diagnostic. All primary systems operational.'
  );

  const [workPerformed, setWorkPerformed] = useState(
    'Replaced engine oil with 5W-30 synthetic, replaced oil filter, and checked brake pad wear.'
  );

  const relatedParts = partsRequests.filter((p) => p.appointmentId === appointment?.id);

  const handleSaveInspection = (e) => {
    e.preventDefault();
    updateAppointmentStatus(appointment.id, appointment.status, {
      inspectionChecklist: checklist,
      inspectionNotes
    });
    setFeedback('Digital inspection checklist saved successfully.');
```

```
    setFeedback('');
    setTimeout(() => setFeedback(''), 3000);
  };

  if (!appointment) return <div className="p-8 text-center text-slate-500">Service order not found.</div>;

  return (
    <div className="space-y-6">
      <PageHeader
        title={`Service Job #${appointment.id}`}
        subtitle={`#${appointment.vehicleName} &bull;`}
        breadcrumbs={[
          { label: 'Assigned Services', link: '/technician/assigned' },
          { label: `Job #${appointment.id}` }
        ]}
        actions={
          <div className="flex gap-2">
            <button onClick={() => setIsPartsModalOpen(true)}
              className="btn btn-secondary btn-sm flex items-center gap-1">
              <Plus size={14} /> Request Parts
            </button>
            {appointment.status !== 'COMPLETED' && (
              <button onClick={() =>
                handleAdvance('COMPLETED')}
                className="btn btn-success btn-sm flex items-center gap-1">
                <CheckCircle2 size={14} /> Complete Job
              </button>
            )}
          </div>
        }
      />
      {feedback && (
        <div className="p-3 bg-emerald-50 text-emerald-700 text-xs rounded border border-emerald-200 flex items-center gap-2">
          <CheckCircle2 size={16} /> <span>{feedback}</span>
        </div>
      )}
      <div className="bg-white p-5 rounded-lg border border-slate-200 shadow-sm space-y-4">
        <div className="flex flex-wrap justify-between items-center gap-2">
          <h3 className="font-bold text-sm text-slate-900">Service Stage Progression</h3>
          <div className="flex gap-1.5 flex-wrap">
            {[ 'VEHICLE_RECEIVED', 'INSPECTION', 'IN_PROGRESS', 'QUALITY_CHECK', 'COMPLETED' ].map((st) => (
              <button key={st} onClick={() =>
                handleAdvance(st)}
                className="btn btn-secondary btn-sm text-[11px] py-1">
                {st.replace('_', ' ')}
            </button>
          )]}
          </div>
        </div>
        <ServiceTimeline currentStatus={appointment.status} />
      </div>
      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        <div className="bg-white p-5 rounded-lg border border-slate-200 shadow-sm space-y-4">
          <h3 className="font-bold text-sm text-slate-900 flex items-center gap-2">
            <ShieldCheck size={18} className="text-emerald-600" /> Digital Multi-Point Inspection
          </h3>
          <form onSubmit={handleSaveInspection}
            className="space-y-3 text-xs">
            <div className="grid grid-cols-2 gap-3">
              {Object.keys(checklist).map((key) => (
                <div key={key}>
                  <label className="block font-semibold mb-1 capitalize text-slate-700">{key.replace(/([A-Z])/g, '$1')}</label>
                  <input
                    type="text"
                    value={checklist[key]}
                    onChange={(e) => setChecklist({
                      ...checklist, [key]: e.target.value
                    })}
                    className="w-full p-2 border rounded text-
```

```

xs"
    />
  </div>
  )))
</div>
</div>

<div>
  <label className="block font-semibold mb-
1 text-slate-700">Diagnostic Findings & Notes</label>
  <textarea
    rows={3}
    value={inspectionNotes}
    onChange={ (e) =>
setInspectionNotes(e.target.value)}
    className="w-full p-2 border rounded
text-xs"
  />
</div>

  <button type="submit" className="w-full btn
btn-primary py-2 font-bold">
    Save Inspection Findings
  </button>
</form>
</div>

  <div className="space-y-6">
    <div className="bg-white p-5 rounded-lg
border border-slate-200 shadow-sm space-y-3">
      <h3 className="font-bold text-sm text-
slate-900 flex items-center gap-2">
        <FileText size={18} className="text-blue-
600" /> Work Performed Log
      </h3>
      <textarea
        rows={4}
        value={workPerformed}
        onChange={ (e) =>
setWorkPerformed(e.target.value)}
        className="w-full p-2 border rounded
text-xs"
      />
      <button
        onClick={() => { setFeedback('Work
performed log updated. '); setTimeout(() =>
setFeedback(''), 3000); }}
        className="btn btn-secondary btn-sm w-
full"
      >
        Update Work Log
      </button>
    </div>

    <div className="bg-white p-5 rounded-lg
border border-slate-200 shadow-sm space-y-3">
      <div className="flex justify-between items-
center">
        <h3 className="font-bold text-sm text-
slate-900 flex items-center gap-2">
          <Boxes size={18} className="text-
purple-600" /> Requisitioned Parts
          ({relatedParts.length})
        </h3>
        <button onClick={() =>
setIsPartsModalOpen(true)} className="btn btn-primary
btn-sm text-xs">
          + Request
        </button>
      </div>
      {relatedParts.map((pr) => (
        <div key={pr.id} className="flex justify-
between items-center p-2.5 bg-slate-50 border rounded
text-xs">
          <div>
            <strong className="text-slate-
800">{pr.partName}</strong>
            <div className="text-slate-500">Qty:
{pr.requestedQuantity} &bull; Urgency:
{pr.urgency}</div>
          </div>
          <span className={`px-2 py-0.5 rounded
font-bold text-[10px] ${pr.status === 'ISSUED' ? 'bg-
emerald-50 text-emerald-700' : 'bg-amber-50 text-amber-
700'}`}>
            {pr.status}
          </span>
        </div>
      )

```

```

    </div>
  )))
</div>
</div>

```

```

<RequestSparePartsModal
  isOpen={isPartsModalOpen}
  onClose={() => setIsPartsModalOpen(false)}
  appointmentId={appointment.id}
  vehicleInfo={appointment.vehicleName}
  technicianId={currentUser?.id}
  technicianName={currentUser?.name}
/>
</div>
);

```

12. src/pages/inventory/InventoryPartsRequests.jsx — Technician Parts Requisitions

```

"use client";

import React, { useState } from 'react';
import { ClipboardList, Check, X, CheckCircle2, Boxes } from
' lucide-react';
import { useData } from '../context/DataContext';
import { PageHeader } from
' ../../components/common/PageHeader';
import { SearchBar } from
' ../../components/common/SearchBar';
import { DataTable } from
' ../../components/common/DataTable';

export const InventoryPartsRequests = () => {
  const { partsRequests, spareParts, approvePartsRequest,
rejectPartsRequest, issueParts } = useData();

  const [searchQuery, setSearchQuery] = useState('');
  const [statusFilter, setStatusFilter] = useState('ALL');
  const [feedback, setFeedback] = useState('');

  const filtered = (partsRequests || []).filter((r) => {
    const q = searchQuery.toLowerCase();
    const matchesSearch =
      r.id.toLowerCase().includes(q) ||
      r.partName?.toLowerCase().includes(q) ||
      r.technicianName?.toLowerCase().includes(q) ||
      r.vehicleInfo?.toLowerCase().includes(q);

    const matchesStatus = statusFilter === 'ALL' || r.status
=== statusFilter;
    return matchesSearch && matchesStatus;
  });

  const handleApprove = (reqId) => {
    approvePartsRequest(reqId);
    setFeedback('Parts requisition approved. ');
    setTimeout(() => setFeedback(''), 3000);
  };

  const handleIssue = (reqId) => {
    issueParts(reqId);
    setFeedback('Part issued to bay and stock deducted
automatically. ');
    setTimeout(() => setFeedback(''), 3000);
  };

  const handleReject = (reqId) => {
    rejectPartsRequest(reqId);
    setFeedback('Parts request rejected. ');
    setTimeout(() => setFeedback(''), 3000);
  };

  const columns = [
    { header: 'Requisition ID', accessor: 'id', render: (row)
=> <span className="font-mono font-bold text-blue-
600">#{row.id}</span> },
    {
      header: 'Part Requested',
      accessor: 'partName',
      render: (row) => (
        <div>
          <strong className="text-slate-
900">{row.partName}</strong>
          <div className="text-xs text-slate-500">SKU:
{row.partNumber}</div>
        </div>
      )

```

```

    )
  },
  { header: 'Qty', accessor: 'requestedQuantity',
    render: (row) => <strong>{row.requestedQuantity}
  pcs</strong> },
  {
    header: 'Stock Status',
    accessor: 'partId',
    render: (row) => {
      const item = spareParts.find((p) => p.id ===
row.partId);
      const inStock = item ? item.availableQuantity :
0;
      const sufficient = inStock >=
row.requestedQuantity;
      return (
        <span className={`font-bold text-xs
${sufficient ? 'text-emerald-600' : 'text-rose-600'}}`>
          {inStock} units {sufficient ? 'Sufficient'
: 'Low Stock'}}
        </span>
      );
    }
  },
  {
    header: 'Target Job',
    accessor: 'vehicleInfo',
    render: (row) => (
      <div className="text-xs">
        <div>{row.vehicleInfo}</div>
        <span className="text-blue-600 font-mono">Job
#{row.appointmentId}</span>
      </div>
    )
  },
  { header: 'Technician', accessor: 'technicianName'
},
  {
    header: 'Status',
    accessor: 'status',
    render: (row) => (
      <span className={`px-2 py-0.5 rounded text-
[10px] font-bold ${
        row.status === 'ISSUED' ? 'bg-emerald-50
text-emerald-700' :
        row.status === 'APPROVED' ? 'bg-blue-50 text-
blue-700' :
        row.status === 'REJECTED' ? 'bg-rose-50 text-
rose-700' : 'bg-amber-50 text-amber-700'
      }}`>
        {row.status}
      </span>
    )
  },
  {
    header: 'Actions',
    align: 'right',
    render: (row) => {
      const item = spareParts.find((p) => p.id ===
row.partId);
      const hasStock = item && item.availableQuantity
>= row.requestedQuantity;

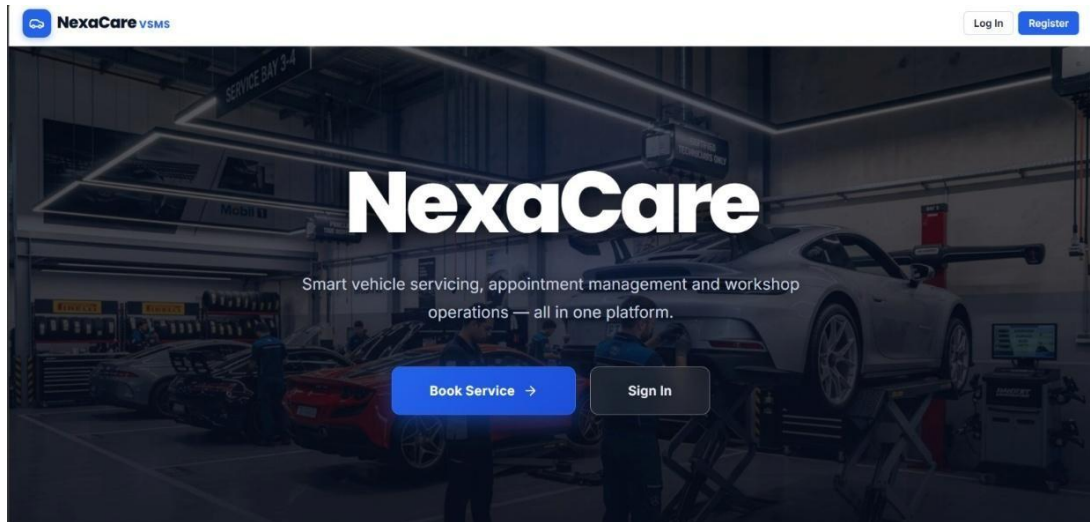
      return (
        <div className="flex gap-1.5 justify-end">
          {row.status === 'PENDING' && (
            <button onClick={() =>
handleApprove(row.id)} className="btn btn-primary btn-
sm text-xs">
              Approve
            </button>
          )}
          {hasStock && row.status !== 'ISSUED' && (
            <button onClick={() =>
handleIssue(row.id)} className="btn btn-success btn-sm
text-xs flex items-center gap-1">
              <Check size={12} /> Issue
            </button>
          )}
          {row.status !== 'ISSUED' && row.status !==
'REJECTED' && (
            <button onClick={() =>
handleReject(row.id)} className="btn btn-danger btn-sm
text-xs">
              <X size={12} />
            </button>
          )
        </div>
      );
    }
  }
];

return (
  <div className="space-y-6">
    <PageHeader
      title="Technician Parts Requisitions Queue"
      subtitle="Approve or reject workshop bay requisitions
and issue components with automated inventory updates"
    />
    {feedback && (
      <div className="p-3 bg-emerald-50 text-emerald-700
text-xs rounded border border-emerald-200 flex items-center
gap-2">
        <CheckCircle2 size={16} /> <span>{feedback}</span>
      </div>
    )}
    <div className="flex gap-4">
      <div className="flex-1">
        <SearchBar value={searchQuery}
onChange={setSearchQuery} placeholder="Search requisitions by
part, technician, vehicle, or ID..." />
      <div>
        <select value={statusFilter} onChange={(e) =>
setStatusFilter(e.target.value)} className="px-3 py-2 border
rounded text-xs bg-white">
          <option value="ALL">All Requisition
Statuses</option>
          <option value="PENDING">PENDING</option>
          <option value="APPROVED">APPROVED</option>
          <option value="ISSUED">ISSUED</option>
          <option value="REJECTED">REJECTED</option>
        </select>
      </div>
      <div className="bg-white rounded-lg border border-slate-
200 overflow-hidden">
        <DataTable columns={columns} data={filtered}
pageSize={8} />
      </div>
    </div>
  );
};

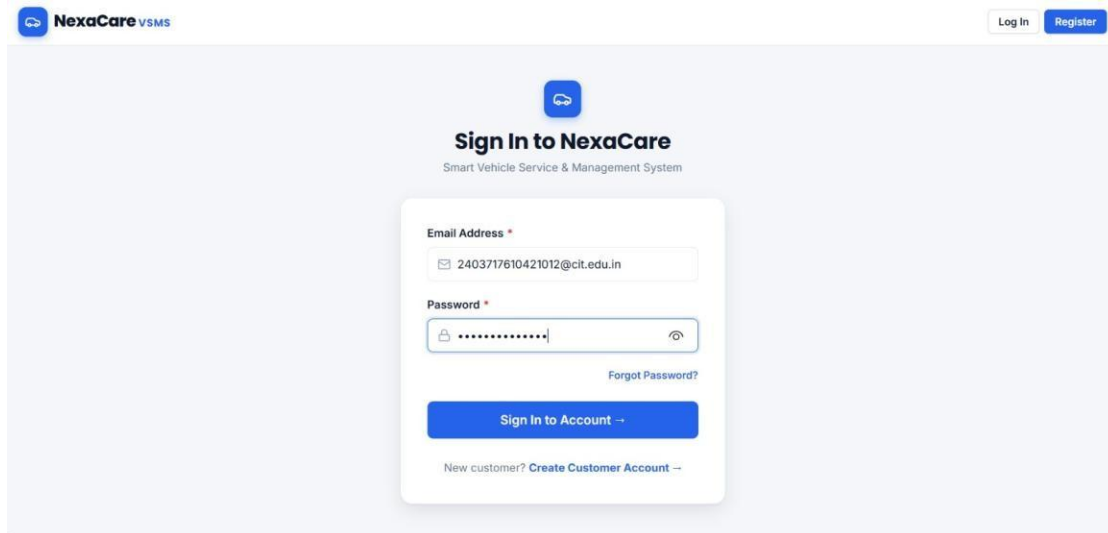
```

DEMONSTRATION:

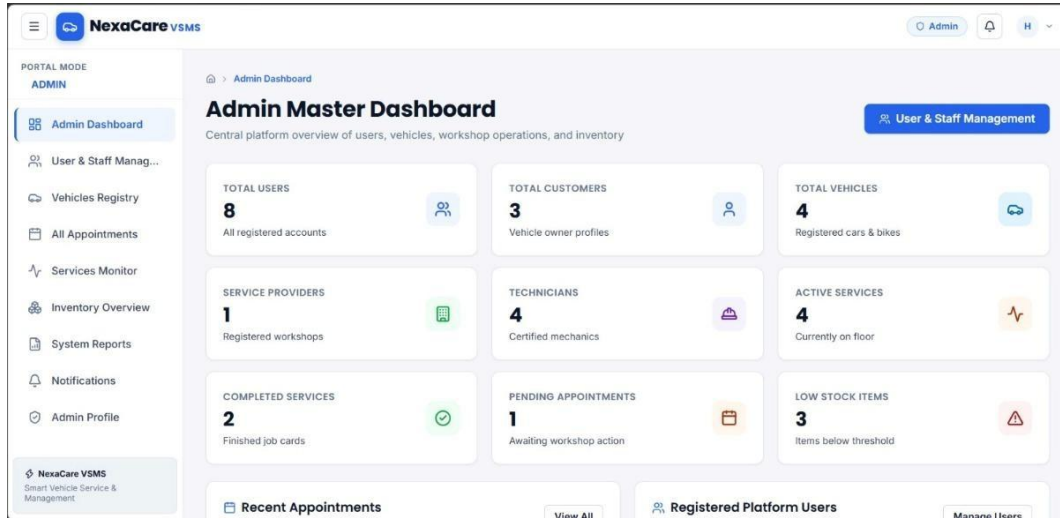
LANDING PAGE:



LOGIN PAGE:



ADMIN DASHBOARD:



SERVICE BOOKING:

The Book Service at NexaCare interface allows users to schedule a vehicle maintenance or repair appointment. It includes a sidebar with navigation options: Dashboard, My Vehicles, Book Service, My Appointments, Service Tracking, Service History, Service Center, Reminders, Notifications, and Profile. The main content area displays the booking form and a summary:

Book Service at NexaCare
Schedule a vehicle maintenance or repair appointment at NexaCare Service Center

NexaCare Service Center
142, Avinashi Road, Peelamedu

Select Vehicle *
No vehicles registered yet. Click here to add one

Service Package Type *
General Service

Appointment Date *
01-10-2026

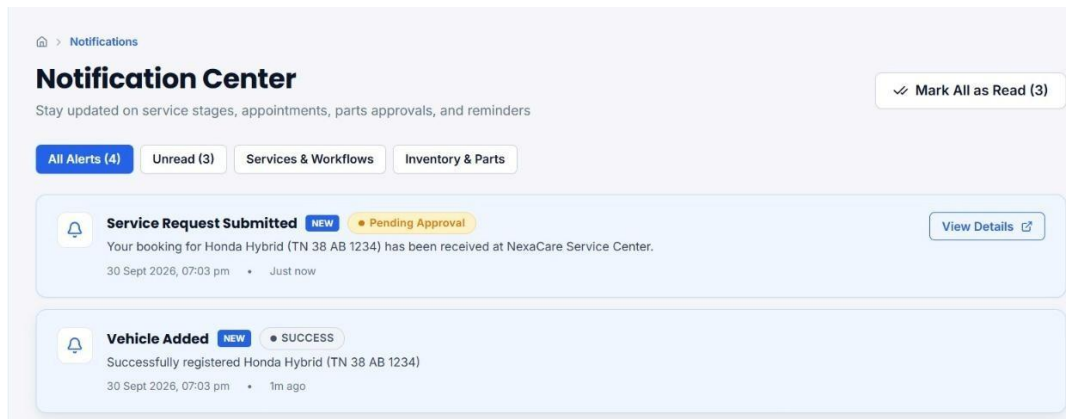
Preferred Time Slot *
10:00 AM

Problem Description / Complaints *
e.g. Brake noise while reversing, routine 10,000 km engine oil and filter replacement...

Booking Summary

Service Package:	General Service
Vehicle:	Not selected
Service Center:	NexaCare Service Center
Date & Slot:	2026-10-01 (10:00 AM)
Estimated Cost:	₹1,800

NOTIFICATION:



TECHNICIAN DASHBOARD:

PORTAL MODE

TECHNICIAN

Dashboard

Assigned Services 2

Request Parts

Parts Status

Completed Services

My Profile

NexaCare VSMS

Smart Vehicle Service & Management

Technician Bay

Technician Bay Portal • Navin

Active workshop service jobs, digital inspection checklists, and parts requisitions

+ Request Spare Parts

View All Assigned Jobs

ASSIGNED JOBS

4

Vehicles in bay

UNDER INSPECTION

0

Multi-point diagnostic

PARTS PENDING

0

Awaiting store issuance

COMPLETED SERVICES

1

Quality tested

ACTIVE JOB #6f0b-f040-1e3f-4912-aae4-40ff7b63c05d9 Pending Approval

Honda Hybrid (TN 38 AB 1234) • AC Service

Customer: Ravi (9089786879) • Scheduled: 3 Oct 2026

Customer Reported Issue: AC is not working

Digital Inspection & Work Log →

Advance Stage →

Assigned Work Queue

Manage service progress step by step

View All (4)

CUSTOMER DASHBOARD:

CUSTOMER

Dashboard

My Vehicles

Book Service

My Appointments

Service Tracking

Service History

Service Center

Reminders 2

Notifications

Profile

NexaCare VSMS

Smart Vehicle Service & Management

Welcome, Ravi

NexaCare Customer Portal • Manage vehicles, track ongoing services, and schedule maintenance

+ Book Service

Service Center

MY VEHICLES

1

Registered cars & bikes

ACTIVE SERVICES

1

Currently at NexaCare

COMPLETED SERVICES

0

Past service history

REMINDERS

0

PUC & service due

ONGOING SERVICE Pending Approval

Honda Hybrid (TN 38 AB 1234) • AC Service

Service Center: NexaCare Service Center • Scheduled: 3 Oct 2026 (11:30 AM)

Track Progress →

Recent Appointments

Upcoming and latest bookings at NexaCare

View All

Honda Hybrid (TN 38 AB 1234)

AC Service • 3 Oct 2026

Pending Approval

My Garage

Registered vehicles

Manage

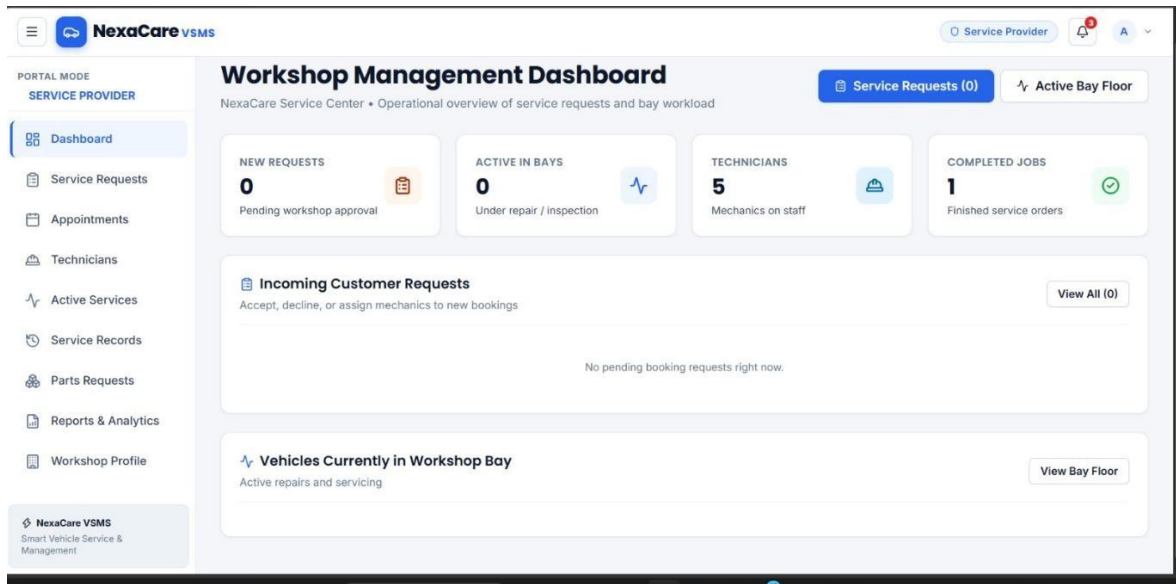
Honda Hybrid

Petrol • 15,000 km • Last: 30 Jul 2026

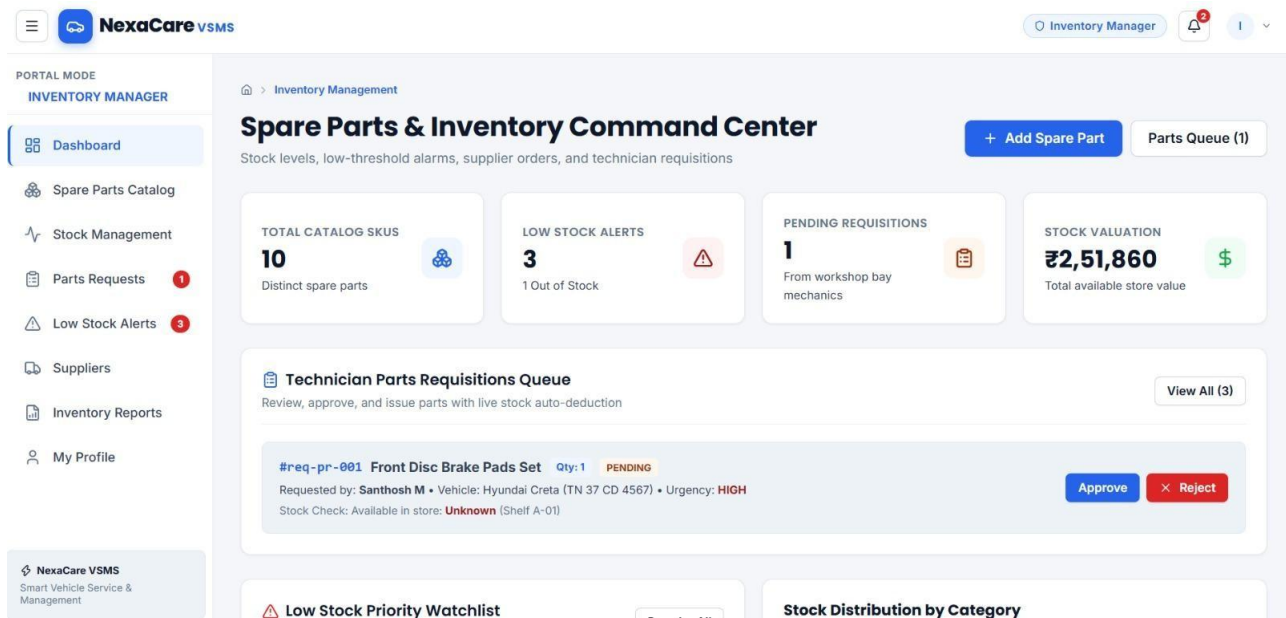
TN 38 AB 1234

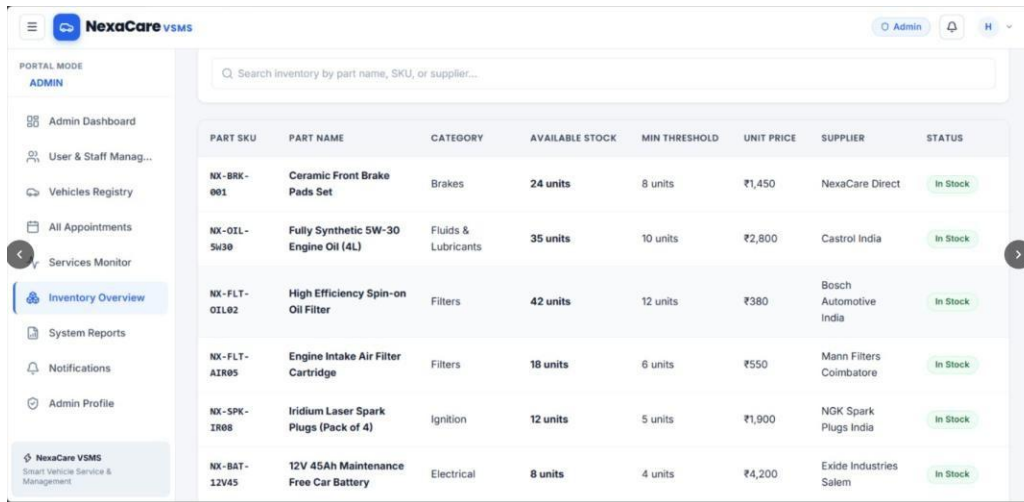
+ Book Service at NexaCare →

SERVICE PROVIDER DASHBOARD:

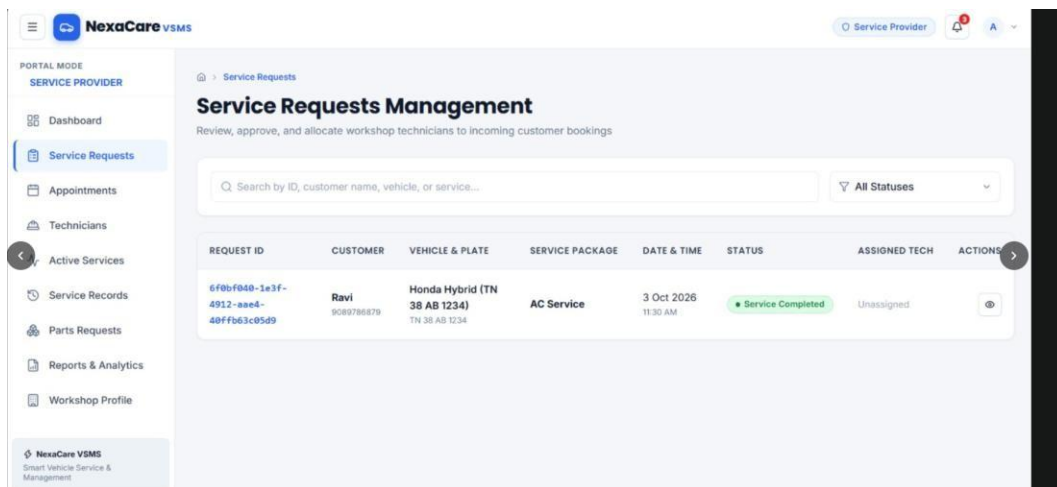


INVENTORY MANAGER DASHBOARD:

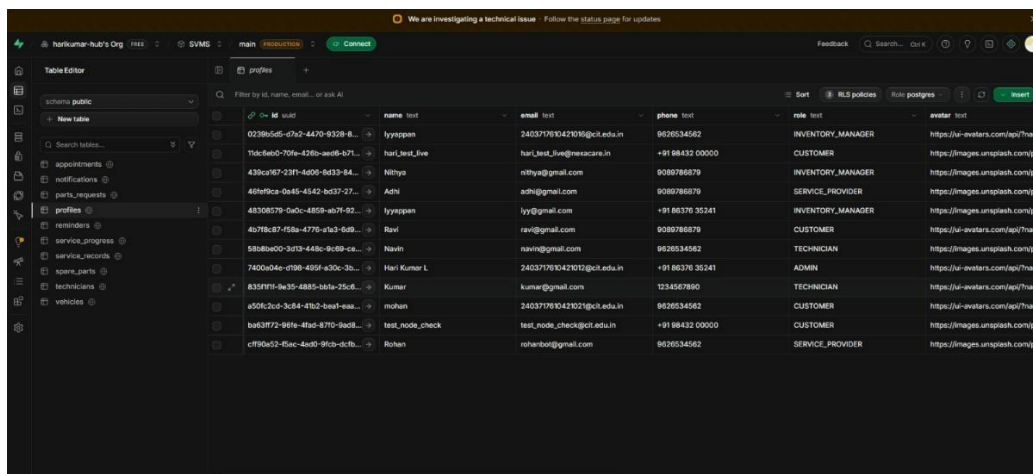


INVENTORY:

PART SKU	PART NAME	CATEGORY	AVAILABLE STOCK	MIN THRESHOLD	UNIT PRICE	SUPPLIER	STATUS
NX-BRK-001	Ceramic Front Brake Pads Set	Brakes	24 units	8 units	₹1,450	NexaCare Direct	In Stock
NX-OIL-5W30	Fully Synthetic 5W-30 Engine Oil (4L)	Fluids & Lubricants	35 units	10 units	₹2,800	Castrol India	In Stock
NX-FLT-OIL02	High Efficiency Spin-on Oil Filter	Filters	42 units	12 units	₹380	Bosch Automotive India	In Stock
NX-FLT-AIR05	Engine Intake Air Filter Cartridge	Filters	18 units	6 units	₹550	Mann Filters Coimbatore	In Stock
NX-SPK-IR08	Iridium Laser Spark Plugs (Pack of 4)	Ignition	12 units	5 units	₹1,900	NGK Spark Plugs India	In Stock
NX-BAT-12V45	12V 45Ah Maintenance Free Car Battery	Electrical	8 units	4 units	₹4,200	Exide Industries Salem	In Stock

SERVICE REQUEST MANAGEMENT PAGE:

REQUEST ID	CUSTOMER	VEHICLE & PLATE	SERVICE PACKAGE	DATE & TIME	STATUS	ASSIGNED TECH	ACTIONS
6F9bF040-1e3f-4912-aae4-40fffb63c05d9	Ravi 9089786879	Honda Hybrid (TN 38 AB 1234) TN 38 AB 1234	AC Service	3 Oct 2026 11:30 AM	Service Completed	Unassigned	

SCHEMA:

id	name	email	phone	role	avatar
0238b5d5-e7d2-4470-8328-8...	lyyasppan	240377970427016@ict.edu.in	9826334562	INVENTORY_MANAGER	https://ui-avatars.com/api/?name=lyyasppan
71dc8e00-709a-428b-ae08-871...	har_test_five	har_test_five@nexacare.in	+91 98432 00000	CUSTOMER	https://images.unsplash.com/photo-1506764197020-000000000000
439cd167-23f1-4a20-8a23-84...	nithya	nithya@gmail.com	9089786879	INVENTORY_MANAGER	https://images.unsplash.com/photo-1506764197020-000000000000
409ef0ca-0a45-4542-b037-27...	Adhi	adhi@gmail.com	9089786879	SERVICE_PROVIDER	https://images.unsplash.com/photo-1506764197020-000000000000
48308579-0a0c-4859-ab7f-82...	lyyasppan	lyy@gmail.com	+91 86376 35241	INVENTORY_MANAGER	https://images.unsplash.com/photo-1506764197020-000000000000
4b78b67-7f5a-4776-e1a3-6a9...	Ravi	ravi@gmail.com	9089786879	CUSTOMER	https://ui-avatars.com/api/?name=Ravi
5848b9d0-3af3-448c-9c09-ca...	Navin	navin@gmail.com	9826334562	TECHNICIAN	https://images.unsplash.com/photo-1506764197020-000000000000
7400a04e-cf98-4809-e30c-3b...	Haril Kumar L	240377970427016@ict.edu.in	+91 98432 00000	ADMIN	https://ui-avatars.com/api/?name=Haril Kumar L
835f7f11-9a35-4885-8a1a-25c...	Kumar	kumar@gmail.com	1234567890	TECHNICIAN	https://ui-avatars.com/api/?name=Kumar
a50f2c0c-3c84-4702-ba8f-aa...	mohan	240377970427016@ict.edu.in	9826334562	CUSTOMER	https://ui-avatars.com/api/?name=mohan
baf37f72-9e1a-4f4d-8710-9ac...	test_node_check	test_node_check@ict.edu.in	+91 98432 00000	CUSTOMER	https://images.unsplash.com/photo-1506764197020-000000000000
c7f9a52-45ec-4a0d-9f03-d3b...	Rohan	rohan@gmail.com	9826334562	SERVICE_PROVIDER	https://images.unsplash.com/photo-1506764197020-000000000000

E-RESOURCE:

GITHUB : <https://github.com/harikumar-hub/SVMS.git>

Appendix C: To Be Determined List

- **TBD-1:** *Final service categories and service duration values.*
- **TBD-2:** *Final service provider location-search and distance criteria.*
- **TBD-3:** *Final notification and maintenance reminder schedules.*
- **TBD-4:** *Final inventory low-stock threshold and replenishment rules.*
- **TBD-5:** *Final report formats and export requirements.*
- **TBD-6:** *Final deployment configuration and production hosting details.*
- **TBD-7:** *Final configuration of future Spring Boot REST API integration.*
- **TBD-8:** *Final configuration of additional notification channels if required.*